

[illegible]


```

}

public class OllamaModelDetails
{
    [JsonPropertyName("family")] public string Family
    { get; set; } = string.Empty;
    [JsonPropertyName("parameter_size")] public string ParameterSize
    { get; set; } = string.Empty;
    [JsonPropertyName("quantization_level")] public string
    QuantizationLevel { get; set; } = string.Empty;
}

public class OllamaChatRequest
{
    [JsonPropertyName("model")] public string Model { get; set; } =
    string.Empty;
    [JsonPropertyName("messages")] public List<OllamaChatMessage>
    Messages { get; set; } = new();
    [JsonPropertyName("stream")] public bool Stream { get; set; } =
    true;
    [JsonPropertyName("options")] public OllamaOptions? Options { get;
    set; }
}

public class OllamaChatMessage
{
    [JsonPropertyName("role")] public string Role { get; set; } =
    string.Empty;
    [JsonPropertyName("content")] public string Content { get; set; } =
    string.Empty;
}

public class OllamaOptions
{
    [JsonPropertyName("temperature")] public float Temperature { get;
    set; } = 0.7f;
    [JsonPropertyName("num_predict")] public int NumPredict { get;
    set; } = 2048;
    [JsonPropertyName("top_p")] public float TopP { get;
    set; } = 0.9f;
    [JsonPropertyName("top_k")] public int TopK { get;
    set; } = 40;
}

public class OllamaChatResponse
{
    [JsonPropertyName("model")] public string Model { get; set; } =
    string.Empty;
    [JsonPropertyName("message")] public OllamaChatMessage? Message {
    get; set; }
    [JsonPropertyName("done")] public bool Done { get; set; }
    [JsonPropertyName("done_reason")] public string? DoneReason { get;
    set; }
}

```



```

public class ChatMessage
{
    public Guid Id { get; set; } = Guid.NewGuid();
    public string Role { get; set; } = string.Empty;
    public string Content { get; set; } = string.Empty;
    public DateTime Timestamp { get; set; } = DateTime.Now;
    public bool IsStreaming { get; set; } = false;
    public string? ModelName { get; set; }
    public long? TokenCount { get; set; }
    public double? GenerationMs { get; set; }
    public bool IsUser => Role == "user";
    public bool IsAssistant => Role == "assistant";
}

public class AppSettings
{
    public string OllamaBaseUrl { get; set; } =
"http://localhost:11434";
    public string DefaultModel { get; set; } = string.Empty;
    public string DefaultSystemPrompt { get; set; } = "You are a
helpful, honest, and harmless AI assistant.";
    public float Temperature { get; set; } = 0.7f;
    public int MaxTokens { get; set; } = 2048;
    public float TopP { get; set; } = 0.9f;
    public int TopK { get; set; } = 40;
    public string Theme { get; set; } = "Dark";
    public bool StreamResponses { get; set; } = true;
    public bool SendWithEnter { get; set; } = true;
    public int FontSize { get; set; } = 14;
    public bool HasSeenTour { get; set; } = false;
    public string CopilotModel { get; set; } = string.Empty;
}

public class PromptTemplate
{
    public string Id { get; set; } = Guid.NewGuid().ToString();
    public string Name { get; set; } = string.Empty;
    public string Category { get; set; } = "General";
    public string Content { get; set; } = string.Empty;
    public string Icon { get; set; } = "\uE8BD";
    public DateTime Created { get; set; } = DateTime.UtcNow;
}

public class Persona
{
    public string Id { get; set; } = Guid.NewGuid().ToString();
    public string Name { get; set; } = string.Empty;
    public string Description { get; set; } = string.Empty;
    public string SystemPrompt { get; set; } = string.Empty;
    public string Avatar { get; set; } = "\uE77B";
    public string AvatarColor { get; set; } = "#8A63FF";
    public string PreferredModel { get; set; } = string.Empty;
    public DateTime Created { get; set; } = DateTime.UtcNow;
}

```

```

public class ChainNode
{
    public string Id          { get; set; } = Guid.NewGuid().ToString();
    public int    Order       { get; set; }
    public string Name        { get; set; } = "Step";
    public string Prompt      { get; set; } = string.Empty;
    public string Model       { get; set; } = string.Empty;
    public string OutputLabel { get; set; } = string.Empty;
    public string? LastOutput { get; set; }
    public bool   IsRunning   { get; set; }
    public bool   IsDone      { get; set; }
}

public class PromptChain
{
    public string Id          { get; set; } = Guid.NewGuid().ToString();
    public string Name        { get; set; } = "New Chain";
    public List<ChainNode> Nodes { get; set; } = new();
    public DateTime Created   { get; set; } = DateTime.UtcNow;
}

public class BatchJob
{
    public string Model       { get; set; } = string.Empty;
    public string Prompt      { get; set; } = string.Empty;
    public string? Response   { get; set; }
    public bool   IsRunning   { get; set; }
    public bool   IsDone      { get; set; }
    public double Ms          { get; set; }
    public string Status      { get; set; } = "Waiting";
}

public class PerformanceSample
{
    public DateTime Timestamp      { get; set; } = DateTime.UtcNow;
    public double   TokensPerSec   { get; set; }
    public double   LatencyMs      { get; set; }
    public string   ModelName      { get; set; } = string.Empty;
    public int      TokensGenerated { get; set; }
}

public class SessionStats
{
    public int      TotalMessages      { get; set; }
    public int      TotalTokens        { get; set; }
    public double   AvgLatencyMs       { get; set; }
    public double   AvgTokensPerSec    { get; set; }
    public double   UptimeMinutes      { get; set; }
    public double   EstimatedSavingsUsd { get; set; }
}

'@ -Encoding UTF8
Write-Ok "Models\OllamaModels.cs"

```



```

private void Load()
{
    if (!File.Exists(_path)) return;
    try { Current =
JsonSerializer.Deserialize<AppSettings>(File.ReadAllText(_path)) ??
new(); } catch { Current = new(); }
}
public void Save()
{
    File.WriteAllText(_path, JsonSerializer.Serialize(Current, new
JsonSerializerOptions { WriteIndented = true }));
    SettingsChanged?.Invoke(this, EventArgs.Empty);
}
}
'@ -Encoding UTF8

Set-Content "$root\Services\ThemeService.cs" '@
using Microsoft.UI.Xaml;
namespace OllamaHub.Services;
public class ThemeService
{
    private readonly SettingsService _s;
    public ThemeService(SettingsService s) { _s = s; }
    public void Apply(Window w)
    {
        if (w.Content is FrameworkElement fe)
            fe.RequestedTheme = _s.Current.Theme == "Light" ?
ElementTheme.Light : ElementTheme.Dark;
    }
}
'@ -Encoding UTF8

Set-Content "$root\Services\OllamaService.cs" '@
using System.Net.Http.Json;
using System.Runtime.CompilerServices;
using System.Text;
using System.Text.Json;
using OllamaHub.Models;
namespace OllamaHub.Services;
public class OllamaService
{
    private readonly HttpClient _http;
    private readonly SettingsService _settings;
    public OllamaService(SettingsService settings)
    {
        _settings = settings;
        _http = new HttpClient { Timeout = TimeSpan.FromMinutes(10) };
        UpdateBaseUrl();
        _settings.SettingsChanged += (_, _) => UpdateBaseUrl();
    }
    private void UpdateBaseUrl()
        => _http.BaseAddress = new
Uri(_settings.Current.OllamaBaseUrl.TrimEnd('/') + "/");
    public async Task<bool> IsOnlineAsync(CancellationToken ct = default)

```

```

        {
            try { return (await _http.GetAsync("api/tags",
ct)).IsSuccessStatusCode; } catch { return false; }
        }
        public async Task<List<OllamaModelInfo>>
GetModelsAsync(CancellationToken ct = default)
        {
            try { return (await
_http.GetFromJsonAsync<OllamaTagsResponse>("api/tags", ct)).Models ??
new(); }
            catch { return new(); }
        }
        public async IEnumerable<string>
ChatStreamAsync(OllamaChatRequest request,
[EnumeratorCancellation] CancellationToken ct = default)
        {
            using var content = new
StringContent(JsonSerializer.Serialize(request), Encoding.UTF8,
"application/json");
            using var response = await _http.PostAsync("api/chat", content,
ct);
            response.EnsureSuccessStatusCode();
            using var stream = await response.Content.ReadAsStreamAsync(ct);
            using var reader = new StreamReader(stream);
            while (!reader.EndOfStream && !ct.IsCancellationRequested)
            {
                var line = await reader.ReadLineAsync(ct);
                if (string.IsNullOrEmpty(line)) continue;
                OllamaChatResponse? chunk = null;
                try { chunk =
JsonSerializer.Deserialize<OllamaChatResponse>(line); } catch { continue;
}
                if (chunk?.Message?.Content is { Length: > 0 } text) yield
return text;
                if (chunk?.Done == true) break;
            }
        }
        public async IEnumerable<string> GenerateStreamAsync(string
model, string prompt,
[EnumeratorCancellation] CancellationToken ct = default)
        {
            var req = new OllamaGenerateRequest { Model = model, Prompt =
prompt, Stream = true };
            using var content = new
StringContent(JsonSerializer.Serialize(req), Encoding.UTF8,
"application/json");
            using var response = await _http.PostAsync("api/generate",
content, ct);
            response.EnsureSuccessStatusCode();
            using var stream = await response.Content.ReadAsStreamAsync(ct);
            using var reader = new StreamReader(stream);
            while (!reader.EndOfStream && !ct.IsCancellationRequested)
            {
                var line = await reader.ReadLineAsync(ct);

```

```

        if (string.IsNullOrEmpty(line)) continue;
        OllamaGenerateResponse? chunk = null;
        try { chunk =
JsonSerializer.Deserialize<OllamaGenerateResponse>(line); } catch {
continue; }
        if (!string.IsNullOrEmpty(chunk?.Response)) yield return
chunk.Response;
        if (chunk?.Done == true) break;
    }
}
public async IAsyncEnumerable<OllamaPullResponse>
PullModelAsync(string modelName,
    [EnumeratorCancellation] CancellationToken ct = default)
{
    var req = new OllamaPullRequest { Name = modelName };
    using var content = new
StringContent(JsonSerializer.Serialize(req), Encoding.UTF8,
"application/json");
    using var response = await _http.PostAsync("api/pull", content,
ct);
    response.EnsureSuccessStatusCode();
    using var stream = await response.Content.ReadAsStreamAsync(ct);
    using var reader = new StreamReader(stream);
    while (!reader.EndOfStream && !ct.IsCancellationRequested)
    {
        var line = await reader.ReadLineAsync(ct);
        if (string.IsNullOrEmpty(line)) continue;
        OllamaPullResponse? chunk = null;
        try { chunk =
JsonSerializer.Deserialize<OllamaPullResponse>(line); } catch { continue;
}
        if (chunk != null) yield return chunk;
        if (chunk?.Status == "success") break;
    }
}
public async Task<bool> DeleteModelAsync(string modelName,
CancellationToken ct = default)
{
    try
    {
        var req = new HttpRequestMessage(HttpMethod.Delete,
"api/delete")
        {
            Content = new StringContent(JsonSerializer.Serialize(new
{ name = modelName }), Encoding.UTF8, "application/json")
        };
        return (await _http.SendAsync(req, ct)).IsSuccessStatusCode;
    }
    catch { return false; }
}
}
'@ -Encoding UTF8

```

```
Set-Content "$root\Services\ChatSessionService.cs" '@'
```

```

using System.Text.Json;
using OllamaHub.Models;
namespace OllamaHub.Services;
public class ChatSessionService
{
    private static readonly string SessionsDir =

Path.Combine(Environment.GetFolderPath(Environment.SpecialFolder.Applicat
ionData), "OllamaHub", "sessions");
    public List<ChatSession> Sessions { get; } = new();
    public event EventHandler? SessionsChanged;
    public ChatSessionService() { Directory.CreateDirectory(SessionsDir);
LoadAll(); }
    private void LoadAll()
    {
        Sessions.Clear();
        if (!Directory.Exists(SessionsDir)) return;
        foreach (var f in Directory.GetFiles(SessionsDir,
        "*.json").OrderByDescending(File.GetLastWriteTime))
        {
            try { var s =
JsonSerializer.Deserialize<ChatSession>(File.ReadAllText(f)); if (s !=
null) Sessions.Add(s); } catch { }
        }
    }
    public ChatSession CreateSession(string modelName, string
systemPrompt = "", string? personaId = null)
    {
        var s = new ChatSession { ModelName = modelName, SystemPrompt =
systemPrompt,
        Title = $"Chat {DateTime.Now:MMM d, h:mm tt}", PersonaId =
personaId };
        Sessions.Insert(0, s); Save(s); SessionsChanged?.Invoke(this,
EventArgs.Empty); return s;
    }
    public void Save(ChatSession s)
    {
        try { File.WriteAllText(Path.Combine(SessionsDir,
        $"{s.Id}.json"),
        JsonSerializer.Serialize(s, new JsonSerializerOptions {
WriteIndented = true })); } catch { }
    }
    public void Delete(ChatSession s)
    {
        Sessions.Remove(s);
        var p = Path.Combine(SessionsDir, $"{s.Id}.json");
        if (File.Exists(p)) File.Delete(p);
        SessionsChanged?.Invoke(this, EventArgs.Empty);
    }
    public void UpdateTitle(ChatSession s, string title) { s.Title =
title; Save(s); SessionsChanged?.Invoke(this, EventArgs.Empty); }
}
'@ -Encoding UTF8

```

```

Set-Content "$root\Services\TokenCounterService.cs" @"
using OllamaHub.Models;
namespace OllamaHub.Services;
public class TokenCounterService
{
    public int Estimate(string text) => string.IsNullOrEmpty(text) ? 0 :
(int)Math.Ceiling(text.Length / 4.0);
    public int EstimateMessages(IEnumerable<ChatMessage> msgs) =>
msgs.Sum(m => Estimate(m.Content) + 4);
    public string Format(int t) => t switch { < 1000 => $"{t} tokens", <
10000 => $"{t/1000.0:F1}k tokens", _ => $"{t/1000}k tokens" };
}
"@ -Encoding UTF8

Set-Content "$root\Services\PromptLibraryService.cs" @"
using System.Collections.ObjectModel;
using System.Text.Json;
using OllamaHub.Models;
namespace OllamaHub.Services;
public class PromptLibraryService
{
    private readonly string _path;
    public ObservableCollection<PromptTemplate> Templates { get; } =
new();
    public PromptLibraryService()
    {
        var dir =
Path.Combine(Environment.GetFolderPath(Environment.SpecialFolder.LocalApp
licationData), "OllamaHub");
        Directory.CreateDirectory(dir); _path = Path.Combine(dir,
"prompts.json");
        Load(); if (Templates.Count == 0) SeedDefaults();
    }
    private void Load()
    {
        if (!File.Exists(_path)) return;
        try { var list =
JsonSerializer.Deserialize<List<PromptTemplate>>(File.ReadAllText(_path))
;
            if (list != null) foreach (var t in list) Templates.Add(t);
        } catch { }
    }
    public void Save() => File.WriteAllText(_path,
JsonSerializer.Serialize(Templates.ToList(), new JsonSerializerOptions {
WriteIndented = true }));
    public void Add(PromptTemplate t) { Templates.Add(t); Save(); }
    public void Delete(PromptTemplate t) { Templates.Remove(t); Save(); }
    public void Update() { Save(); }
    private void SeedDefaults()
    {
        var d = new[] {
            new PromptTemplate { Name="Explain Simply",
Category="Learning", Content="Explain simply:\n\n",
Icon="\uE82D" },

```

```

        new PromptTemplate { Name="Code Review",
Category="Coding", Content="Review this code:\n\n```\n\n```,
Icon="\uE943" },
        new PromptTemplate { Name="Write Tests",
Category="Coding", Content="Write unit tests for:\n\n```\n\n```,
Icon="\uE8C4" },
        new PromptTemplate { Name="Summarize",
Category="Writing", Content="Summarize concisely:\n\n",
Icon="\uE8D2" },
        new PromptTemplate { Name="Fix Grammar",
Category="Writing", Content="Fix grammar and clarity:\n\n",
Icon="\uE8AB" },
        new PromptTemplate { Name="Translate",
Category="Language", Content="Translate to English:\n\n",
Icon="\uE8C1" },
        new PromptTemplate { Name="SQL Query",
Category="Coding", Content="Write a SQL query to:\n\n",
Icon="\uE8D7" },
        new PromptTemplate { Name="Brainstorm",
Category="Creative", Content="Brainstorm 10 ideas for:\n\n",
Icon="\uE90F" },
        new PromptTemplate { Name="Debug Error",
Category="Coding", Content="Debug this
error:\n\nError:\n\nCode:\n\n```\n\n```, Icon="\uEBE8" },
        new PromptTemplate { Name="Pros and Cons",
Category="General", Content="List pros and cons of:\n\n",
Icon="\uE8EF" },
    };

```

```

        foreach (var t in d) Templates.Add(t); Save();
    }
}

```

```
'@ -Encoding UTF8
```

```

Set-Content "$root\Services\PersonaService.cs" @"
using System.Collections.ObjectModel;
using System.Text.Json;
using OllamaHub.Models;
namespace OllamaHub.Services;
public class PersonaService
{
    private readonly string _path;
    public ObservableCollection<Persona> Personas { get; } = new();
    public PersonaService()
    {
        var dir =
Path.Combine(Environment.GetFolderPath(Environment.SpecialFolder.LocalApp
licationData), "OllamaHub");
        Directory.CreateDirectory(dir); _path = Path.Combine(dir,
"personas.json");
        Load(); if (Personas.Count == 0) SeedDefaults();
    }
    private void Load()
    {
        if (!File.Exists(_path)) return;

```



```

        try { var list =
JsonSerializer.Deserialize<List<Persona>>(File.ReadAllText(_path));
            if (list != null) foreach (var p in list) Personas.Add(p);
        } catch { }
    }
    public void Save() => File.WriteAllText(_path,
JsonSerializer.Serialize(Personas.ToList(), new JsonSerializerOptions {
WriteIndented = true }));
    public void Add(Persona p)      { Personas.Add(p);      Save(); }
    public void Delete(Persona p) { Personas.Remove(p); Save(); }
    public void Update()           { Save(); }
    private void SeedDefaults()
    {
        var d = new[] {
            new Persona { Name="Code Expert",      Description="Senior
software engineer",      AvatarColor="#7C6AF7", Avatar="\uE943",
            SystemPrompt="You are a senior software engineer with 15
years of experience. Give precise, production-ready code with
explanations. Prefer clean architecture and SOLID principles." },
            new Persona { Name="Blunt Editor",      Description="No-
nonsense writing editor", AvatarColor="#F87171", Avatar="\uE8AB",
            SystemPrompt="You are a brutally honest editor. Cut
unnecessary words, fix clarity issues, and tell the user exactly what is
wrong without sugar-coating it." },
            new Persona { Name="Socratic Teacher", Description="Teaches
by asking questions",   AvatarColor="#60A5FA", Avatar="\uE82D",
            SystemPrompt="You are a Socratic teacher. Instead of
giving answers directly, guide the user to discover them through
thoughtful questions. Be patient and encouraging." },
            new Persona { Name="Creative Writer",   Description="Imaginative storyteller", AvatarColor="#FB923C",
Avatar="\uE8F4",
            SystemPrompt="You are a creative writer with a vivid
imagination. Help craft compelling narratives, suggest creative ideas,
and bring stories to life with rich descriptive language." },
            new Persona { Name="Data Analyst",      Description="Statistics and data insights", AvatarColor="#4ADE80",
Avatar="\uE8EF",
            SystemPrompt="You are a data analyst who loves finding
patterns and insights. Respond with structured analysis, suggest
visualizations, and always back claims with reasoning." },
        };
        foreach (var p in d) Personas.Add(p); Save();
    }
}
'@ -Encoding UTF8

```

```

Set-Content "$root\Services\ChainService.cs" '@
using System.Collections.ObjectModel;
using System.Text.Json;
using OllamaHub.Models;
namespace OllamaHub.Services;
public class ChainService
{

```

```

        private readonly string _path;
        public ObservableCollection<PromptChain> Chains { get; } = new();
        public ChainService()
        {
            var dir =
Path.Combine(Environment.GetFolderPath(Environment.SpecialFolder.LocalApp
licationData), "OllamaHub");
            Directory.CreateDirectory(dir); _path = Path.Combine(dir,
"chains.json");
            Load(); if (Chains.Count == 0) SeedDefault();
        }
        private void Load()
        {
            if (!File.Exists(_path)) return;
            try { var list =
JsonSerializer.Deserialize<List<PromptChain>>(File.ReadAllText(_path));
                if (list != null) foreach (var c in list) Chains.Add(c); }
catch { }
        }
        public void Save() => File.WriteAllText(_path,
JsonSerializer.Serialize(Chains.ToList(), new JsonSerializerOptions {
WriteIndented = true }));
        public void Add(PromptChain c) { Chains.Add(c); Save(); }
        public void Delete(PromptChain c) { Chains.Remove(c); Save(); }
        public void Update() { Save(); }
        private void SeedDefault()
        {
            var chain = new PromptChain { Name = "Blog Post Writer" };
            chain.Nodes.Add(new ChainNode { Order=0, Name="Research",
Prompt="Research the topic and list 5 key points about: {{INPUT}}",
OutputLabel="Research notes" });
            chain.Nodes.Add(new ChainNode { Order=1, Name="Outline",
Prompt="Using these research notes, create a blog post
outline:\n\n{{PREV}}", OutputLabel="Outline" });
            chain.Nodes.Add(new ChainNode { Order=2, Name="Write",
Prompt="Write a full blog post based on this outline:\n\n{{PREV}}",
OutputLabel="Draft" });
            chain.Nodes.Add(new ChainNode { Order=3, Name="Polish",
Prompt="Polish and improve this blog post for clarity and
engagement:\n\n{{PREV}}", OutputLabel="Final post" });
            Chains.Add(chain); Save();
        }
    }
}
'@ -Encoding UTF8

Set-Content "$root\Services\ExportService.cs" @"
using System.Text;
using OllamaHub.Models;
namespace OllamaHub.Services;
public class ExportService
{
    public string ToMarkdown(ChatSession s)
    {
        var sb = new StringBuilder();

```

```

        sb.AppendLine($"# {s.Title}");
        sb.AppendLine($"> Model: {s.ModelName} | {s.CreatedAt:yyyy-MM-
dd HH:mm}");
        sb.AppendLine();
        foreach (var m in s.Messages)
        {
            if (m.Role == "system") continue;
            sb.AppendLine(m.Role == "user" ? "## You" : $"## {m.ModelName
?? "Assistant"}");
            sb.AppendLine(m.Content); sb.AppendLine();
        }
        return sb.ToString();
    }
    public string ToPlainText(ChatSession s)
    {
        var sb = new StringBuilder();
        sb.AppendLine($"=== {s.Title} ===");
        sb.AppendLine($"Model: {s.ModelName} | {s.CreatedAt:yyyy-MM-dd
HH:mm}");
        sb.AppendLine(new string('-', 60));
        foreach (var m in s.Messages)
        {
            if (m.Role == "system") continue;
            sb.AppendLine($"[{(m.Role == "user" ? "YOU" :
"ASSISTANT")}]");
            sb.AppendLine(m.Content); sb.AppendLine();
        }
        return sb.ToString();
    }
    public string ToHtml(ChatSession s)
    {
        var sb = new StringBuilder();
        sb.AppendLine($"<!DOCTYPE html><html><head><meta charset='utf-
8'><title>{H(s.Title)}</title>");
        sb.AppendLine($"<style>body{font-family:Segoe UI,sans-serif;max-
width:800px;margin:0 auto;padding:2rem;background:#1e1e2e;color:#cdd6f4}
h1{color:#cba6f7}.meta{color:#6c7086;font-size:.85rem;margin-bottom:2rem}
.msg{margin:1rem 0;padding:1rem 1.25rem;border-radius:12px}
.user{background:#313244;border-left:3px solid #cba6f7}
.assistant{background:#1e1e2e;border-left:3px solid #89dceb;border:1px
solid #313244}
.role{font-size:.75rem;font-weight:600;text-transform:uppercase;letter-
spacing:1px;margin-bottom:.5rem;color:#6c7086}
pre{white-space:pre-wrap;margin:0;font-size:.95rem;line-
height:1.6}</style></head><body>");
        sb.AppendLine($"<h1>{H(s.Title)}</h1><div class='meta'>Model:
{s.ModelName} | {s.CreatedAt:yyyy-MM-dd HH:mm}</div>");
        foreach (var m in s.Messages)
        {
            if (m.Role == "system") continue;
            var cls = m.Role == "user" ? "user" : "assistant";
            var lbl = m.Role == "user" ? "You" : (m.ModelName ??
"Assistant");

```

```

        sb.AppendLine($"<div class='msg {cls}'><div
class='role'>{H(lbl)}</div><pre>{H(m.Content)}</pre></div>");
    }
    sb.AppendLine("</body></html>"); return sb.ToString();
}
private static string H(string s) =>
s.Replace("&", "&amp;").Replace("<", "&lt;").Replace(">", "&gt;").Replace("\
", "&quot;");
public async Task SaveToFileAsync(string content, string defaultName,
string filter)
{
    var picker = new Windows.Storage.Pickers.FileSavePicker();
    picker.SuggestedFileName = defaultName;
    var window = App.Current.MainWindowHandle ?? throw new
InvalidOperationException("No window");
    WinRT.Interop.InitializeWithWindow.Initialize(picker,
WinRT.Interop.WindowNative.GetWindowHandle(window));
    foreach (var f in filter.Split('|')) { var parts = f.Split(':');
if (parts.Length==2) picker.FileTypeChoices.Add(parts[0],
new[] {parts[1]}); }
    var file = await picker.PickSaveFileAsync();
    if (file != null) await
Windows.Storage.FileIO.WriteTextAsync(file, content);
}
}
'@ -Encoding UTF8

```

```

Set-Content "$root\Services\PerformanceService.cs" '@
using System.Collections.ObjectModel;
using OllamaHub.Models;
namespace OllamaHub.Services;
public class PerformanceService
{
    private readonly DateTime _start = DateTime.UtcNow;
    private int _totalTokens, _totalMessages;
    private readonly List<double> _latencies = new(), _tps = new();
    public ObservableCollection<PerformanceSample> RecentSamples { get; }
= new();
    public void RecordGeneration(string model, int tokens, double
latencyMs)
    {
        var t = latencyMs > 0 ? tokens / (latencyMs / 1000.0) : 0;
        _totalTokens += tokens; _totalMessages++;
        _latencies.Add(latencyMs); _tps.Add(t);
        var sample = new PerformanceSample { ModelName=model,
TokensGenerated=tokens, LatencyMs=latencyMs, TokensPerSec=t };
        App.UIDispatcher?.TryEnqueue(() => { RecentSamples.Add(sample);
if (RecentSamples.Count > 50) RecentSamples.RemoveAt(0); });
    }
    public SessionStats GetStats() => new()
    {
        TotalMessages=_totalMessages, TotalTokens=_totalTokens,
AvgLatencyMs=_latencies.Count>0?_latencies.Average():0,
AvgTokensPerSec=_tps.Count>0?_tps.Average():0,

```

```

        UptimeMinutes=(DateTime.UtcNow-_start).TotalMinutes,
        EstimatedSavingsUsd=_totalTokens/1_000_000.0*15.0
    };
    public double CurrentTokensPerSec =>
    _tps.Count>0?_tps.TakeLast(5).Average():0;
}
'@ -Encoding UTF8

Set-Content "$root\Services\SlashCommandService.cs" @"
using OllamaHub.Models;
namespace OllamaHub.Services;
public record SlashCommand(string Trigger, string Name, string
Description, string Content);
public class SlashCommandService
{
    private readonly PromptLibraryService _prompts;
    private readonly List<SlashCommand> _builtins = new()
    {
        new("/explain",    "Explain",    "Explain simply",
"Explain the following in simple terms:\n\n"),
        new("/code",      "Code Review", "Review code",
"Review this code for bugs and improvements:\n\n```\n\n`"),
        new("/fix",       "Fix This",    "Debug and fix",
"Debug and fix the following:\n\n"),
        new("/test",      "Write Tests", "Generate unit tests",
"Write comprehensive unit tests for:\n\n```\n\n`"),
        new("/summarize", "Summarize",   "Condense text",
"Summarize the following concisely:\n\n"),
        new("/translate", "Translate",   "Translate to English",
"Translate to English:\n\n"),
        new("/improve",   "Improve",     "Polish prose",
"Improve the clarity and style of:\n\n"),
        new("/brainstorm","Brainstorm",  "Generate ideas",
"Brainstorm 10 creative ideas for:\n\n"),
        new("/pros",      "Pros & Cons", "Weigh a decision",          "List
the pros and cons of:\n\n"),
        new("/eli5",      "ELI5",       "Explain like I'm 5",
"Explain this like I'm 5 years old:\n\n"),
    };
    public SlashCommandService(PromptLibraryService prompts) { _prompts =
prompts; }
    public IEnumerable<SlashCommand> Search(string query)
    {
        var q = query.TrimStart('/').ToLowerInvariant();
        var fromLib = _prompts.Templates
            .Where(t => t.Name.Contains(q,
StringComparison.OrdinalIgnoreCase))
            .Select(t => new SlashCommand("/" +
t.Name.ToLowerInvariant().Replace(" ", ""), t.Name, t.Category,
t.Content));
        return _builtins.Where(c => c.Trigger.Contains(q) ||
c.Name.Contains(q, StringComparison.OrdinalIgnoreCase))
            .Concat(fromLib).Take(8);
    }
}

```

```

}
'@ -Encoding UTF8

Set-Content "$root\Services\VoiceService.cs" @"
using System.Speech.Recognition;
using System.Speech.Synthesis;
namespace OllamaHub.Services;
public class VoiceService : IDisposable
{
    private SpeechSynthesizer? _synth;
    private SpeechRecognitionEngine? _rec;
    private bool _listening, _disposed;
    public bool VoiceOutputEnabled { get; set; } = false;
    public double SpeechRate { get; set; } = 1.0;
    public bool IsListening => _listening;
    public event EventHandler<string>? SpeechRecognized;
    public event EventHandler? ListeningStarted;
    public event EventHandler? ListeningStopped;
    public event EventHandler<string>? ErrorOccurred;
    public void Speak(string text)
    {
        if (!VoiceOutputEnabled || string.IsNullOrEmpty(text) ||
            _disposed) return;
        try { EnsureSynth(); _synth!.SpeakAsyncCancelAll();
            _synth.Rate=(int)Math.Clamp(SpeechRate*5-5,-10,10);
            _synth.SpeakAsync(text.Length>500?text[..500]+"...":text);
        } catch (Exception ex) { Dispatch(() => ErrorOccurred?.Invoke(this,
            ex.Message)); }
    }
    public void StopSpeaking() { try { _synth?.SpeakAsyncCancelAll(); }
    catch { } }
    public void StartListening()
    {
        if (_listening || _disposed) return;
        try
        {
            _rec = new
SpeechRecognitionEngine(System.Globalization.CultureInfo.CurrentCulture);
            _rec.LoadGrammar(new DictationGrammar());
            _rec.SpeechRecognized += (_, e) => Dispatch(() =>
SpeechRecognized?.Invoke(this, e.Result.Text));
            _rec.SetInputToDefaultAudioDevice();
            _rec.RecognizeAsync(RecognizeMode.Multiple);
            _listening = true; Dispatch(() =>
ListeningStarted?.Invoke(this, EventArgs.Empty));
        }
        catch (Exception ex) { Dispatch(() => ErrorOccurred?.Invoke(this,
            $"Mic error: {ex.Message}")); }
    }
    public void StopListening()
    {
        if (!_listening) return;
        try { _rec?.RecognizeAsyncStop(); _rec?.Dispose(); } catch { }
    }
}
"

```



```

using CommunityToolkit.Mvvm.ComponentModel;
using CommunityToolkit.Mvvm.Input;
using OllamaHub.Services;
namespace OllamaHub.ViewModels;
public partial class MainViewModel : BaseViewModel
{
    private readonly OllamaService _ollama;
    [ObservableProperty] private bool _ollamaOnline;
    public MainViewModel(OllamaService ollama)
    {
        _ollama = ollama;
        _ = CheckAsync();
        _ = Task.Run(async () => { while (true) { await
Task.Delay(10000); var on = await _ollama.IsOnlineAsync();
App.UIDispatcher?.TryEnqueue(() => OllamaOnline = on); } });
    }
    [RelayCommand] private async Task CheckAsync() => OllamaOnline =
await _ollama.IsOnlineAsync();
}
'@ -Encoding UTF8

Set-Content "$root\ViewModels\ChatViewModel.cs" @"
using System.Collections.ObjectModel;
using CommunityToolkit.Mvvm.ComponentModel;
using CommunityToolkit.Mvvm.Input;
using OllamaHub.Models;
using OllamaHub.Services;
namespace OllamaHub.ViewModels;
public partial class ChatViewModel : BaseViewModel
{
    private readonly OllamaService _ollama;
    private readonly ChatSessionService _sessions;
    private readonly SettingsService _settings;
    private readonly PerformanceService _perf;
    private readonly TokenCounterService _tokens;
    private readonly PersonaService _personas;
    private CancellationTokenSource? _cts;

    [ObservableProperty] private ObservableCollection<ChatSession>
_chatSessions = new();
    [ObservableProperty] private ChatSession?
_currentSession;
    [ObservableProperty] private ObservableCollection<ChatMessage>
_messages = new();
    [ObservableProperty] private string
_inputText = string.Empty;
    [ObservableProperty] private ObservableCollection<OllamaModelInfo>
_availableModels = new();
    [ObservableProperty] private string
_selectedModel = string.Empty;
    [ObservableProperty] private bool
_isGenerating;
    [ObservableProperty] private string
_statusText = "Ready";

```



```

    [ObservableProperty] private int
_contextTokens;
    [ObservableProperty] private int
_maxContextTokens = 4096;
    [ObservableProperty] private double
_contextPercent;
    [ObservableProperty] private string
_contextColor = "#4ADE80";
    [ObservableProperty] private Persona?
_activPersona;

    public ChatViewModel(OllamaService ollama, ChatSessionService
sessions, SettingsService settings,
                        PerformanceService perf, TokenCounterService
tokens, PersonaService personas)
    {
        _ollama = ollama; _sessions = sessions; _settings = settings;
        _perf = perf; _tokens = tokens; _personas = personas;
        foreach (var s in _sessions.Sessions) ChatSessions.Add(s);
        SelectedModel = _settings.Current.DefaultModel;
        _ = LoadModelsAsync();
    }

    [RelayCommand]
    private async Task LoadModelsAsync()
    {
        var models = await _ollama.GetModelsAsync();
        AvailableModels.Clear();
        foreach (var m in models) AvailableModels.Add(m);
        if (string.IsNullOrEmpty(SelectedModel) && models.Count > 0)
SelectedModel = models[0].Name;
    }

    [RelayCommand]
    private void NewChat()
    {
        if (string.IsNullOrEmpty(SelectedModel)) return;
        var prompt = ActivPersona?.SystemPrompt ??
_settings.Current.DefaultSystemPrompt;
        var s = _sessions.CreateSession(SelectedModel, prompt,
ActivPersona?.Id);
        ChatSessions.Insert(0, s); OpenSession(s);
    }

    [RelayCommand]
    private void OpenSession(ChatSession session)
    {
        CurrentSession = session; Messages.Clear();
        foreach (var m in session.Messages) Messages.Add(m);
        SelectedModel = session.ModelName;
        if (session.PersonaId != null)
            ActivPersona = _personas.Personas.FirstOrDefault(p => p.Id ==
session.PersonaId);
        UpdateContextHealth();
    }

```

```

    }

    [RelayCommand]
    private void DeleteSession(ChatSession session)
    {
        _sessions.Delete(session); ChatSessions.Remove(session);
        if (CurrentSession == session) { CurrentSession = null;
Messages.Clear(); }
    }

    [RelayCommand]
    private async Task SendMessageAsync()
    {
        if (string.IsNullOrEmpty(InputText) || IsGenerating ||
string.IsNullOrEmpty(SelectedModel)) return;
        if (CurrentSession == null)
        {
            var prompt = ActivPersona?.SystemPrompt ??
_settings.Current.DefaultSystemPrompt;
            var s = _sessions.CreateSession(SelectedModel, prompt,
ActivPersona?.Id);
            ChatSessions.Insert(0, s); OpenSession(s);
        }
        var userMsg = new ChatMessage { Role="user",
Content=InputText.Trim() };
        Messages.Add(userMsg); CurrentSession!.Messages.Add(userMsg);
        if (CurrentSession.Messages.Count(m => m.Role=="user") == 1)
            _sessions.UpdateTitle(CurrentSession, InputText.Length > 40 ?
InputText[..40]+"..." : InputText);
        InputText = string.Empty; IsGenerating = true; StatusText =
$"Generating...";
        var assistantMsg = new ChatMessage { Role="assistant",
Content="", IsStreaming=true, ModelName=SelectedModel };
        Messages.Add(assistantMsg);
        CurrentSession.Messages.Add(assistantMsg);
        _cts = new CancellationTokenSource();
        var start = DateTime.UtcNow;
        try
        {
            var history = CurrentSession.Messages.Where(m => m !=
assistantMsg)
                .Select(m => new OllamaChatMessage { Role=m.Role,
Content=m.Content }).ToList();
            if (!string.IsNullOrEmpty(CurrentSession.SystemPrompt))
                history.Insert(0, new OllamaChatMessage { Role="system",
Content=CurrentSession.SystemPrompt });
            var req = new OllamaChatRequest { Model=SelectedModel,
Messages=history, Stream=_settings.Current.StreamResponses,
Options=new OllamaOptions {
Temperature=_settings.Current.Temperature,
NumPredict=_settings.Current.MaxTokens, TopP=_settings.Current.TopP,
TopK=_settings.Current.TopK } };
            await foreach (var token in _ollama.ChatStreamAsync(req,
_cts.Token))

```

```

        {
            assistantMsg.Content += token;
            var idx = Messages.IndexOf(assistantMsg);
            if (idx >= 0) { Messages.RemoveAt(idx);
Messages.Insert(idx, assistantMsg); }
        }
    }
    catch (OperationCanceledException) { assistantMsg.Content +=
"\n\n[Stopped]"; }
    catch (Exception ex) { assistantMsg.Content =
$"Error: {ex.Message}"; }
    finally
    {
        var ms = (DateTime.UtcNow - start).TotalMilliseconds;
        assistantMsg.IsStreaming = false; assistantMsg.GenerationMs =
ms;
        assistantMsg.TokenCount =
_tokens.Estimate(assistantMsg.Content);
        var idx2 = Messages.IndexOf(assistantMsg);
        if (idx2 >= 0) { Messages.RemoveAt(idx2);
Messages.Insert(idx2, assistantMsg); }
        _perf.RecordGeneration(SelectedModel,
(int) (assistantMsg.TokenCount??0), ms);
        _sessions.Save(CurrentSession!); IsGenerating = false;
        StatusText = $"Done in {ms:F0}ms Â·
{_tokens.Format((int) (assistantMsg.TokenCount??0))}";
        UpdateContextHealth();
    }
}

[RelayCommand] private void StopGeneration() => _cts?.Cancel();

[RelayCommand]
private void ClearCurrentChat()
{
    if (CurrentSession == null) return;
    CurrentSession.Messages.Clear(); Messages.Clear();
_sessions.Save(CurrentSession); UpdateContextHealth();
}

private void UpdateContextHealth()
{
    ContextTokens = _tokens.EstimateMessages(Messages);
    ContextPercent = Math.Min(100.0, ContextTokens /
(double)MaxContextTokens * 100.0);
    ContextColor = ContextPercent < 60 ? "#4ADE80" : ContextPercent
< 85 ? "#FB923C" : "#F87171";
}

partial void OnSelectedModelChanged(string value) { if
(CurrentSession != null) CurrentSession.ModelName = value; }
}
'@ -Encoding UTF8

```

```

Set-Content "$root\ViewModels\CopilotViewModel.cs" @"
using System.Collections.ObjectModel;
using CommunityToolkit.Mvvm.ComponentModel;
using CommunityToolkit.Mvvm.Input;
using OllamaHub.Models;
using OllamaHub.Services;
namespace OllamaHub.ViewModels;
public partial class CopilotViewModel : BaseViewModel
{
    private readonly OllamaService _ollama;
    private readonly SettingsService _settings;
    private CancellationTokenSource? _cts;

    [ObservableProperty] private ObservableCollection<OllamaModelInfo>
_availableModels = new();
    [ObservableProperty] private string _selectedModel = string.Empty;
    [ObservableProperty] private string _inputText = string.Empty;
    [ObservableProperty] private string _codeInput = string.Empty;
    [ObservableProperty] private string _outputText = string.Empty;
    [ObservableProperty] private bool _isGenerating;
    [ObservableProperty] private string _activeMode = "Chat";
    [ObservableProperty] private string _statusText = "Ready";
    [ObservableProperty] private ObservableCollection<CopilotMessage>
_messages = new();

    public List<string> Modes { get; } = new() { "Chat", "Explain",
"Review", "Fix", "Tests", "Docs", "Refactor" };

    public CopilotViewModel(OllamaService ollama, SettingsService
settings)
    {
        _ollama = ollama; _settings = settings;
        _ = LoadModelsAsync();
    }

    [RelayCommand]
    private async Task LoadModelsAsync()
    {
        var models = await _ollama.GetModelsAsync();
        AvailableModels.Clear();
        foreach (var m in models) AvailableModels.Add(m);
        var preferred = _settings.Current.CopilotModel;
        if (!string.IsNullOrEmpty(preferred) && models.Any(m => m.Name ==
preferred))
            SelectedModel = preferred;
        else if (models.Count > 0)
            SelectedModel = models.FirstOrDefault(m =>
m.Name.Contains("coder") || m.Name.Contains("code"))?.Name ??
models[0].Name;
    }

    private string BuildPrompt()
    {
        return ActiveMode switch

```

```

        {
            "Explain" => $"Explain this code clearly, step by
step:\n\n```\n{CodeInput}\n```,
            "Review" => $"Review this code for bugs, performance
issues, and best practices. Be specific:\n\n```\n{CodeInput}\n```,
            "Fix" => $"Fix this code. Show the corrected version
with a brief explanation of what was
wrong:\n\n```\n{CodeInput}\n```\n\nError/Issue: {InputText}",
            "Tests" => $"Write comprehensive unit tests for this code.
Include happy path, edge cases, and failure
cases:\n\n```\n{CodeInput}\n```,
            "Docs" => $"Write complete XML documentation comments for
every method, property, and class in this
code:\n\n```\n{CodeInput}\n```,
            "Refactor" => $"Refactor this code to be cleaner, more
readable, and follow best practices. Keep the same
behavior:\n\n```\n{CodeInput}\n```,
            - => InputText // Chat mode
        };
    };
}

```

```

[RelayCommand]
private async Task RunAsync()
{
    if (IsGenerating) { _cts?.Cancel(); return; }
    var prompt = BuildPrompt();
    if (string.IsNullOrEmpty(prompt)) return;

    // Add user message for chat mode
    if (ActiveMode == "Chat")
        Messages.Add(new CopilotMessage { Role = "user", Content =
InputText });

    IsGenerating = true; StatusText = "Thinking...";
    OutputText = string.Empty;
    _cts = new CancellationTokenSource();
    var response = new System.Text.StringBuilder();
    try
    {
        await foreach (var token in
_ollama.GenerateStreamAsync(SelectedModel, prompt, _cts.Token))
        {
            response.Append(token);
            OutputText = response.ToString();
        }
        if (ActiveMode == "Chat")
            Messages.Add(new CopilotMessage { Role = "assistant",
Content = response.ToString(), Model = SelectedModel });
    }
    catch (OperationCanceledException) { OutputText += "\n[Stopped]";
}

    catch (Exception ex) { OutputText = $"Error: {ex.Message}"; }
    finally
    {

```

```

        IsGenerating = false;
        StatusText = "Done";
        InputText = string.Empty;
    }
}

[RelayCommand] private void Stop() => _cts?.Cancel();

[RelayCommand]
private void ClearChat() { Messages.Clear(); OutputText =
string.Empty; }
}

public class CopilotMessage
{
    public string Role { get; set; } = string.Empty;
    public string Content { get; set; } = string.Empty;
    public string? Model { get; set; }
    public bool IsUser => Role == "user";
    public bool IsAssistant => Role == "assistant";
}
'@ -Encoding UTF8

Set-Content "$root\ViewModels\PersonaViewModel.cs" @"
using System.Collections.ObjectModel;
using CommunityToolkit.Mvvm.ComponentModel;
using CommunityToolkit.Mvvm.Input;
using OllamaHub.Models;
using OllamaHub.Services;
namespace OllamaHub.ViewModels;
public partial class PersonaViewModel : BaseViewModel
{
    private readonly PersonaService _svc;
    private readonly OllamaService _ollama;
    public ObservableCollection<Persona> Personas => _svc.Personas;
    [ObservableProperty] private Persona? _selected;
    [ObservableProperty] private string _name = string.Empty;
    [ObservableProperty] private string _description = string.Empty;
    [ObservableProperty] private string _systemPrompt = string.Empty;
    [ObservableProperty] private string _avatarColor = "#7C6AF7";
    [ObservableProperty] private ObservableCollection<OllamaModelInfo>
_availableModels = new();
    [ObservableProperty] private string _preferredModel = string.Empty;
    public List<string> AvatarColors { get; } = new() {
"#7C6AF7", "#60A5FA", "#4ADE80", "#FB923C", "#F87171", "#F472B6", "#2DD4BF", "#F
BBF24" };
    public PersonaViewModel(PersonaService svc, OllamaService ollama)
    {
        _svc = svc; _ollama = ollama; _ = LoadModelsAsync();
    }
    private async Task LoadModelsAsync()
    {
        var models = await _ollama.GetModelsAsync();

```

```

        AvailableModels.Clear(); foreach (var m in models)
AvailableModels.Add(m);
    }
    [RelayCommand]
    private void Save()
    {
        if (string.IsNullOrEmpty(Name)) return;
        if (Selected != null)
        {
            Selected.Name = Name; Selected.Description = Description;
            Selected.SystemPrompt = SystemPrompt; Selected.AvatarColor =
AvatarColor;
            Selected.PreferredModel = PreferredModel; _svc.Update();
        }
        else
        {
            _svc.Add(new Persona { Name=Name, Description=Description,
SystemPrompt=SystemPrompt, AvatarColor=AvatarColor,
PreferredModel=PreferredModel });
        }
    }
    [RelayCommand] private void Delete() { if (Selected != null)
_svc.Delete(Selected); Selected=null; Clear(); }
    [RelayCommand]
    private void New() { Selected=null; Clear(); }
    private void Clear() {
Name=Description=SystemPrompt=PreferredModel=string.Empty;
AvatarColor="#7C6AF7"; }
    public void SelectPersona(Persona p)
    {
        Selected = p; Name = p.Name; Description = p.Description;
        SystemPrompt = p.SystemPrompt; AvatarColor = p.AvatarColor;
PreferredModel = p.PreferredModel;
    }
}
'@ -Encoding UTF8

Set-Content "$root\ViewModels\ChainViewModel.cs" '@'
using System.Collections.ObjectModel;
using CommunityToolkit.Mvvm.ComponentModel;
using CommunityToolkit.Mvvm.Input;
using OllamaHub.Models;
using OllamaHub.Services;
namespace OllamaHub.ViewModels;
public partial class ChainViewModel : BaseViewModel
{
    private readonly ChainService _svc;
    private readonly OllamaService _ollama;
    public ObservableCollection<PromptChain> Chains => _svc.Chains;
    [ObservableProperty] private PromptChain? _selectedChain;
    [ObservableProperty] private ObservableCollection<ChainNode> _nodes =
new();
    [ObservableProperty] private bool _isRunning;

```

```

    [ObservableProperty] private string _statusText = "Select a chain or
create a new one";
    [ObservableProperty] private string _initialInput = string.Empty;
    [ObservableProperty] private ObservableCollection<OllamaModelInfo>
_availableModels = new();

    public ChainViewModel(ChainService svc, OllamaService ollama)
    {
        _svc = svc; _ollama = ollama; _ = LoadModelsAsync();
    }

    private async Task LoadModelsAsync()
    {
        var models = await _ollama.GetModelsAsync();
        AvailableModels.Clear(); foreach (var m in models)
AvailableModels.Add(m);
    }

    public void SelectChain(PromptChain chain)
    {
        SelectedChain = chain; Nodes.Clear();
        foreach (var n in chain.Nodes.OrderBy(n => n.Order))
Nodes.Add(n);
    }

    [RelayCommand]
    private async Task RunChainAsync()
    {
        if (SelectedChain == null || IsRunning) return;
        IsRunning = true;
        string prevOutput = InitialInput;
        foreach (var node in Nodes)
        {
            node.IsRunning = true; node.IsDone = false; node.LastOutput =
string.Empty;
            StatusText = $"Running: {node.Name}...";
            var prompt = node.Prompt
                .Replace("{INPUT}", InitialInput)
                .Replace("{PREV}", prevOutput);
            var model = string.IsNullOrEmpty(node.Model) &&
AvailableModels.Count > 0
                ? AvailableModels[0].Name : node.Model;
            var sb = new System.Text.StringBuilder();
            try
            {
                await foreach (var token in
_ollama.GenerateStreamAsync(model, prompt))
                {
                    sb.Append(token);
                    node.LastOutput = sb.ToString();
                    var idx = Nodes.IndexOf(node);
                    if (idx >= 0) { Nodes.RemoveAt(idx);
Nodes.Insert(idx, node); }
                }
            }
        }
    }

```



```

        prevOutput = sb.ToString();
    }
    catch (Exception ex) { node.LastOutput = $"Error:
{ex.Message}"; }
    node.IsRunning = false; node.IsDone = true;
    var idx2 = Nodes.IndexOf(node);
    if (idx2 >= 0) { Nodes.RemoveAt(idx2); Nodes.Insert(idx2,
node); }
    }
    IsRunning = false; StatusText = "Chain complete!";
    _svc.Update();
}

[RelayCommand]
private void AddNode()
{
    if (SelectedChain == null) return;
    var node = new ChainNode { Order = Nodes.Count, Name = $"Step
{Nodes.Count+1}",
    Model = AvailableModels.Count > 0 ? AvailableModels[0].Name :
"" };
    SelectedChain.Nodes.Add(node); Nodes.Add(node); _svc.Update();
}

[RelayCommand]
private void RemoveNode(ChainNode node)
{
    if (SelectedChain == null) return;
    SelectedChain.Nodes.Remove(node); Nodes.Remove(node);
    _svc.Update();
}

[RelayCommand]
private void NewChain()
{
    var chain = new PromptChain { Name = "New Chain" };
    _svc.Add(chain); SelectChain(chain);
}

[RelayCommand]
private void DeleteChain() { if (SelectedChain != null) {
    _svc.Delete(SelectedChain); SelectedChain=null; Nodes.Clear(); } }
}
'@ -Encoding UTF8

```

```

Set-Content "$root\ViewModels\HistoryViewModel.cs" '@'
using System.Collections.ObjectModel;
using CommunityToolkit.Mvvm.ComponentModel;
using CommunityToolkit.Mvvm.Input;
using OllamaHub.Models;
using OllamaHub.Services;
namespace OllamaHub.ViewModels;
public partial class HistoryViewModel : ObservableObject
{

```

```

        private readonly ChatSessionService _sessions;
        [ObservableProperty] private ObservableCollection<ChatSession>
        _filteredSessions = new();
        [ObservableProperty] private string _searchQuery = string.Empty;
        public HistoryViewModel(ChatSessionService sessions)
        {
            _sessions = sessions;
            _sessions.SessionsChanged += (_, _) =>
App.UIDispatcher?.TryEnqueue(Refresh);
            Refresh();
        }
        partial void OnSearchQueryChanged(string value) => Refresh();
        public void Refresh()
        {
            FilteredSessions.Clear();
            var q = _sessions.Sessions.AsEnumerable();
            if (!string.IsNullOrEmpty(SearchQuery))
                q = q.Where(s => s.Title.Contains(SearchQuery,
StringComparison.OrdinalIgnoreCase) ||
                    s.Messages.Any(m => m.Content.Contains(SearchQuery,
StringComparison.OrdinalIgnoreCase)));
            foreach (var s in q.OrderByDescending(s => s.LastMessageAt))
                FilteredSessions.Add(s);
        }
        [RelayCommand] private void DeleteChat(ChatSession s) {
        _sessions.Delete(s); FilteredSessions.Remove(s); }
    }
    '@ -Encoding UTF8

```

```

Set-Content "$root\ViewModels\ModelsViewModel.cs" '@'
using System.Collections.ObjectModel;
using CommunityToolkit.Mvvm.ComponentModel;
using CommunityToolkit.Mvvm.Input;
using OllamaHub.Models;
using OllamaHub.Services;
namespace OllamaHub.ViewModels;
public partial class ModelsViewModel : BaseViewModel
{
    private readonly OllamaService _ollama;
    [ObservableProperty] private ObservableCollection<OllamaModelInfo>
    _models = new();
    [ObservableProperty] private string _pullModelName = string.Empty;
    [ObservableProperty] private string _pullStatus = string.Empty;
    [ObservableProperty] private double _pullProgress = 0;
    [ObservableProperty] private bool _isPulling;
    [ObservableProperty] private bool _isLoading;
    private CancellationTokenSource? _cts;
    public ModelsViewModel(OllamaService ollama) { _ollama = ollama; _ =
LoadModelsAsync(); }
    [RelayCommand] private async Task LoadModelsAsync() { IsLoading=true;
var m=await _ollama.GetModelsAsync(); Models.Clear(); foreach(var x in m)
Models.Add(x); IsLoading=false; }
    [RelayCommand] private async Task PullModelAsync()
    {

```

```

        if (string.IsNullOrEmpty(PullModelName) || IsPulling) return;
        IsPulling=true; PullStatus="Starting..."; PullProgress=0;
        _cts=new CancellationTokenSource();
        try
        {
            await foreach (var chunk in
                _ollama.PullModelAsync(PullModelName.Trim(), _cts.Token))
            {
                PullStatus=chunk.Status;
                if (chunk.Total.HasValue && chunk.Total > 0)
                PullProgress=(double) (chunk.Completed??0)/chunk.Total.Value*100;
            }
            PullStatus="Done!"; PullModelName=string.Empty; await
LoadModelsAsync();
        }
        catch (OperationCanceledException) { PullStatus="Cancelled"; }
        catch (Exception ex) { PullStatus=$"Error: {ex.Message}"; }
        finally { IsPulling=false; PullProgress=0; }
    }
    [RelayCommand] private async Task DeleteModelAsync(OllamaModelInfo m)
{ if (await _ollama.DeleteModelAsync(m.Name)) Models.Remove(m); }
    [RelayCommand] private void CancelPull() => _cts?.Cancel();
}
'@ -Encoding UTF8

```

```

Set-Content "$root\ViewModels\SettingsViewModel.cs" '@
using CommunityToolkit.Mvvm.ComponentModel;
using CommunityToolkit.Mvvm.Input;
using OllamaHub.Services;
namespace OllamaHub.ViewModels;
public partial class SettingsViewModel : BaseViewModel
{
    private readonly SettingsService _settings;
    [ObservableProperty] private string _ollamaUrl;
    [ObservableProperty] private string _defaultModel;
    [ObservableProperty] private string _systemPrompt;
    [ObservableProperty] private float _temperature;
    [ObservableProperty] private int _maxTokens;
    [ObservableProperty] private bool _streamResponses;
    [ObservableProperty] private string _copilotModel;
    [ObservableProperty] private string _savedMessage = string.Empty;
    public SettingsViewModel(SettingsService settings)
    {
        _settings=settings; _ollamaUrl=settings.Current.OllamaBaseUrl;
        _defaultModel=settings.Current.DefaultModel;
        _systemPrompt=settings.Current.DefaultSystemPrompt;
        _temperature=settings.Current.Temperature;
        _maxTokens=settings.Current.MaxTokens;
        _streamResponses=settings.Current.StreamResponses;
        _copilotModel=settings.Current.CopilotModel;
    }
    [RelayCommand] private void Save()
    {

```

```

        _settings.Current.OllamaBaseUrl=OllamaUrl;
        _settings.Current.DefaultModel=DefaultModel;
        _settings.Current.DefaultSystemPrompt=SystemPrompt;
        _settings.Current.Temperature=Temperature;
        _settings.Current.MaxTokens=MaxTokens;
        _settings.Current.StreamResponses=StreamResponses;
        _settings.Current.CopilotModel=CopilotModel; _settings.Save();
        SavedMessage="Saved!";
        =
        Task.Delay(2000).ContinueWith(_=>App.UIDispatcher?.TryEnqueue(()=>SavedMe
        ssage=string.Empty));
    }
}
'@ -Encoding UTF8

```

```

Set-Content "$root\ViewModels\PerformanceViewModel.cs" '@
using System.Collections.ObjectModel;
using CommunityToolkit.Mvvm.ComponentModel;
using CommunityToolkit.Mvvm.Input;
using OllamaHub.Models;
using OllamaHub.Services;
namespace OllamaHub.ViewModels;
public partial class PerformanceViewModel : BaseViewModel
{
    private readonly PerformanceService _perf;
    private readonly OllamaService _ollama;
    private readonly System.Timers.Timer _ticker;
    [ObservableProperty] private double _currentTps;
    [ObservableProperty] private double _avgLatencyMs;
    [ObservableProperty] private int _totalTokens;
    [ObservableProperty] private int _totalMessages;
    [ObservableProperty] private double _uptimeMinutes;
    [ObservableProperty] private double _estimatedSavings;
    [ObservableProperty] private bool _ollamaOnline;
    [ObservableProperty] private ObservableCollection<PerformanceSample>
    _samples = new();
    public PerformanceViewModel(PerformanceService perf, OllamaService
    ollama)
    {
        _perf=perf; _ollama=ollama;
        _ticker=new System.Timers.Timer(2000);
        _ticker.Elapsed+=(_,_)=>Refresh(); _ticker.Start(); Refresh();
    }
    private void Refresh()
    {
        var stats=_perf.GetStats();
        App.UIDispatcher?.TryEnqueue(()=>
        {
            CurrentTps=Math.Round(_perf.CurrentTokensPerSec,1);
            AvgLatencyMs=Math.Round(stats.AvgLatencyMs,0);
            TotalTokens=stats.TotalTokens;
            TotalMessages=stats.TotalMessages;
            UptimeMinutes=Math.Round(stats.UptimeMinutes,1);
            EstimatedSavings=Math.Round(stats.EstimatedSavingsUsd,4);

```

```

        Samples.Clear(); foreach(var s in
_perf.RecentSamples.TakeLast(20)) Samples.Add(s);
    });
}
[RelayCommand] private async Task CheckStatusAsync() => OllamaOnline
= await _ollama.IsOnlineAsync();
}
'@ -Encoding UTF8

```

```

Set-Content "$root\ViewModels\PromptLibraryViewModel.cs" '@
using System.Collections.ObjectModel;
using CommunityToolkit.Mvvm.ComponentModel;
using CommunityToolkit.Mvvm.Input;
using OllamaHub.Models;
using OllamaHub.Services;
namespace OllamaHub.ViewModels;
public partial class PromptLibraryViewModel : BaseViewModel
{
    private readonly PromptLibraryService _svc;
    public ObservableCollection<PromptTemplate> Templates =>
    _svc.Templates;
    [ObservableProperty] private string _newName="",
    _newCategory="General", _newContent="", _searchText="";
    [ObservableProperty] private PromptTemplate? _selected;
    [ObservableProperty] private ObservableCollection<PromptTemplate>
    _filtered = new();
    public List<string> Categories { get; } = new() {
    "General","Coding","Writing","Learning","Language","Creative","Other" };
    public PromptLibraryViewModel(PromptLibraryService svc) { _svc=svc;
    RefreshFiltered(); }
    partial void OnSearchTextChanged(string v) => RefreshFiltered();
    public void RefreshFiltered()
    {
        Filtered.Clear(); var q=SearchText.Trim().ToLowerInvariant();
        foreach(var t in Templates)
        if(string.IsNullOrEmpty(q) || t.Name.Contains(q,StringComparison.OrdinalIgn
oreCase) || t.Category.Contains(q,StringComparison.OrdinalIgnoreCase))
        Filtered.Add(t);
    }
    [RelayCommand] private void AddTemplate() {
    if(string.IsNullOrEmpty(NewName) || string.IsNullOrEmpty(NewConte
nt)) return; _svc.Add(new
    PromptTemplate{Name=NewName.Trim(),Category=NewCategory,Content=NewConten
t.Trim()}); NewName=NewContent=string.Empty; RefreshFiltered(); }
    [RelayCommand] private void DeleteTemplate(PromptTemplate t) {
    _svc.Delete(t); RefreshFiltered(); }
    [RelayCommand] private void SaveSelected() {
    if(Selected!=null){_svc.Update();RefreshFiltered();} }
}
'@ -Encoding UTF8

```

```

Set-Content "$root\ViewModels\ModelCompareViewModel.cs" '@
using System.Collections.ObjectModel;
using CommunityToolkit.Mvvm.ComponentModel;

```

```

using CommunityToolkit.Mvvm.Input;
using OllamaHub.Models;
using OllamaHub.Services;
namespace OllamaHub.ViewModels;
public partial class ModelCompareViewModel : BaseViewModel
{
    private readonly OllamaService _ollama;
    [ObservableProperty] private ObservableCollection<OllamaModelInfo>
_availableModels = new();
    [ObservableProperty] private string
_modelA="", _modelB="", _prompt="", _responseA="", _responseB="";
    [ObservableProperty] private bool _isRunning;
    [ObservableProperty] private double _msA, _msB;
    [ObservableProperty] private int _tokensA, _tokensB;
    public ModelCompareViewModel(OllamaService ollama) { _ollama=ollama;
    _=LoadModelsAsync(); }
    private async Task LoadModelsAsync() { var m=await
_ollama.GetModelsAsync(); AvailableModels.Clear(); foreach(var x in m)
AvailableModels.Add(x); if(m.Count>0) ModelA=m[0].Name; if(m.Count>1)
ModelB=m[1].Name; }
    [RelayCommand] private async Task RunCompareAsync()
    {

if(string.IsNullOrEmpty(Prompt)||string.IsNullOrEmpty(ModelA)||
string.IsNullOrEmpty(ModelB)) return;
        IsRunning=true; ResponseA=ResponseB=string.Empty; MsA=MsB=0;
TokensA=TokensB=0;
        await
Task.WhenAll(RunModelAsync(ModelA,t=>ResponseA+=t, (ms,tok)=>{MsA=ms;Token
sA=tok;}),
RunModelAsync(ModelB,t=>ResponseB+=t, (ms,tok)=>{MsB=ms;TokensB=tok;}));
        IsRunning=false;
    }
    private async Task RunModelAsync(string model,Action<string>
onToken,Action<double,int> onDone)
    {
        var start=DateTime.UtcNow; int tokens=0;
        var req=new OllamaChatRequest{Model=model,Messages=new
List<OllamaChatMessage>{new(){Role="user",Content=Prompt}},Stream=true};
        try { await foreach(var tok in
_ollama.ChatStreamAsync(req)){App.UIDispatcher?.TryEnqueue(()=>onToken(to
k));tokens+=(int)Math.Ceiling(tok.Length/4.0);} }
        catch(Exception
ex){App.UIDispatcher?.TryEnqueue(()=>onToken($"\\n[Error:
{ex.Message}]")); }
        onDone((DateTime.UtcNow-start).TotalMilliseconds,tokens);
    }
    [RelayCommand] private void
ClearAll(){Prompt=ResponseA=ResponseB=string.Empty;MsA=MsB=0;TokensA=Toke
nsB=0;}
}
'@ -Encoding UTF8

```

```
Set-Content "$root\\ViewModels\\TerminalViewModel.cs" '@'
```



```

Write-Step "Writing App.xaml + App.xaml.cs"
Set-Content "$root\App.xaml" @"
<?xml version="1.0" encoding="utf-8"?>
<Application x:Class="OllamaHub.App"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml">
    <Application.Resources>
        <ResourceDictionary>
            <ResourceDictionary.MergedDictionaries>
                <XamlControlsResources xmlns="using:Microsoft.UI.Xaml.Controls"/>
                <ResourceDictionary Source="Styles/AppStyles.xaml"/>
            </ResourceDictionary.MergedDictionaries>
        </ResourceDictionary>
    </Application.Resources>
</Application>
"@ -Encoding UTF8

```

```

Set-Content "$root\App.xaml.cs" @"
using Microsoft.Extensions.DependencyInjection;
using Microsoft.UI.Dispatching;
using Microsoft.UI.Xaml;
using OllamaHub.Services;
using OllamaHub.ViewModels;
namespace OllamaHub;
public partial class App : Application
{
    public static IServiceProvider Services { get; private set; } =
    null!;
    public static new App Current =>
    (App)Application.Current;
    public static DispatcherQueue? UIDispatcher { get; private set; }
    private Window? _window;
    public Window? MainWindowHandle => _window;
    public App() { InitializeComponent(); Services = ConfigureServices();
    }

    private static IServiceProvider ConfigureServices()
    {
        var s = new ServiceCollection();
        s.AddSingleton<OllamaService>();
        s.AddSingleton<ChatSessionService>();
        s.AddSingleton<SettingsService>();
        s.AddSingleton<ThemeService>();
        s.AddSingleton<PromptLibraryService>();
        s.AddSingleton<PersonaService>();
        s.AddSingleton<ChainService>();
        s.AddSingleton<ExportService>();
        s.AddSingleton<TokenCounterService>();
        s.AddSingleton<PerformanceService>();
        s.AddSingleton<VoiceService>();
        s.AddSingleton<SlashCommandService>();
        s.AddSingleton<ShareService>();
        s.AddTransient<MainViewModel>();
        s.AddTransient<ChatViewModel>();
        s.AddTransient<CopilotViewModel>();
    }
}

```



```

        <LinearGradientBrush StartPoint="0,0" EndPoint="1,1">
            <GradientStop Color="#7C6AF7" Offset="0"/>
            <GradientStop Color="#60A5FA" Offset="1"/>
        </LinearGradientBrush>
    </Border.Background>
    <FontIcon Glyph="&#xE99A;" FontSize="20" Foreground="White"
        HorizontalAlignment="Center"
VerticalAlignment="Center"/>
    </Border>
    <StackPanel HorizontalAlignment="Center" Spacing="2">
        <Ellipse x:Name="StatusDot" Width="7" Height="7"
Fill="#45475A" HorizontalAlignment="Center"/>
        <TextBlock x:Name="StatusLabel" Text="..." FontSize="8"
Foreground="#45475A"
            HorizontalAlignment="Center" MaxWidth="60"
TextTrimming="CharacterEllipsis"/>
    </StackPanel>
</StackPanel>

    <!-- Nav buttons -->
    <StackPanel Grid.Row="1" Spacing="2" Margin="6,0">
        <Button x:Name="BtnChat" Tag="Chat"
Click="NavBtn_Click" Width="56" Height="56" Padding="0"
HorizontalAlignment="Center" Background="Transparent" BorderThickness="0"
CornerRadius="12">
            <ToolTipService.ToolTip><ToolTip Content="Chat â€” talk to
your local AI models"/></ToolTipService.ToolTip>
            <StackPanel Spacing="3" HorizontalAlignment="Center">
                <FontIcon Glyph="&#xE8BD;" FontSize="20"
Foreground="#CDD6F4"/>
                <TextBlock Text="Chat" FontSize="8"
Foreground="#6C7086" HorizontalAlignment="Center"/>
            </StackPanel>
        </Button>
        <Button x:Name="BtnCopilot" Tag="Copilot"
Click="NavBtn_Click" Width="56" Height="56" Padding="0"
HorizontalAlignment="Center" Background="Transparent" BorderThickness="0"
CornerRadius="12">
            <ToolTipService.ToolTip><ToolTip Content="CoPilot â€” AI code
assistant powered by Ollama"/></ToolTipService.ToolTip>
            <StackPanel Spacing="3" HorizontalAlignment="Center">
                <FontIcon Glyph="&#xE943;" FontSize="20"
Foreground="#CDD6F4"/>
                <TextBlock Text="CoPilot" FontSize="8"
Foreground="#6C7086" HorizontalAlignment="Center"/>
            </StackPanel>
        </Button>
        <Button x:Name="BtnModels" Tag="Models"
Click="NavBtn_Click" Width="56" Height="56" Padding="0"
HorizontalAlignment="Center" Background="Transparent" BorderThickness="0"
CornerRadius="12">
            <ToolTipService.ToolTip><ToolTip Content="Models â€” browse,
pull and manage Ollama models"/></ToolTipService.ToolTip>
            <StackPanel Spacing="3" HorizontalAlignment="Center">

```

```

        <FontIcon Glyph="#xE77B;" FontSize="20"
Foreground="#CDD6F4"/>
        <TextBlock Text="Models" FontSize="8"
Foreground="#6C7086" HorizontalAlignment="Center"/>
    </StackPanel>
</Button>
    <Button x:Name="BtnHistory" Tag="History"
Click="NavBtn_Click" Width="56" Height="56" Padding="0"
HorizontalAlignment="Center" Background="Transparent" BorderThickness="0"
CornerRadius="12">
        <ToolTipService.ToolTip><ToolTip Content="History â€” search
and manage past conversations"/></ToolTipService.ToolTip>
        <StackPanel Spacing="3" HorizontalAlignment="Center">
            <FontIcon Glyph="#xE81C;" FontSize="20"
Foreground="#CDD6F4"/>
            <TextBlock Text="History" FontSize="8"
Foreground="#6C7086" HorizontalAlignment="Center"/>
        </StackPanel>
    </Button>
    <Button x:Name="BtnPersonas" Tag="Personas"
Click="NavBtn_Click" Width="56" Height="56" Padding="0"
HorizontalAlignment="Center" Background="Transparent" BorderThickness="0"
CornerRadius="12">
        <ToolTipService.ToolTip><ToolTip Content="Personas â€” custom
AI characters with unique personalities"/></ToolTipService.ToolTip>
        <StackPanel Spacing="3" HorizontalAlignment="Center">
            <FontIcon Glyph="#xE716;" FontSize="20"
Foreground="#CDD6F4"/>
            <TextBlock Text="Personas" FontSize="8"
Foreground="#6C7086" HorizontalAlignment="Center"/>
        </StackPanel>
    </Button>
    <Button x:Name="BtnPrompts" Tag="Prompts"
Click="NavBtn_Click" Width="56" Height="56" Padding="0"
HorizontalAlignment="Center" Background="Transparent" BorderThickness="0"
CornerRadius="12">
        <ToolTipService.ToolTip><ToolTip Content="Prompts â€”
reusable prompt template library (type / in
chat)"/></ToolTipService.ToolTip>
        <StackPanel Spacing="3" HorizontalAlignment="Center">
            <FontIcon Glyph="#xE8F4;" FontSize="20"
Foreground="#CDD6F4"/>
            <TextBlock Text="Prompts" FontSize="8"
Foreground="#6C7086" HorizontalAlignment="Center"/>
        </StackPanel>
    </Button>
    <Button x:Name="BtnChain" Tag="Chain"
Click="NavBtn_Click" Width="56" Height="56" Padding="0"
HorizontalAlignment="Center" Background="Transparent" BorderThickness="0"
CornerRadius="12">
        <ToolTipService.ToolTip><ToolTip Content="Chain â€” visual
prompt pipeline builder (unique feature!)" /></ToolTipService.ToolTip>
        <StackPanel Spacing="3" HorizontalAlignment="Center">

```

```

        <FontIcon Glyph="#xE8EF;" FontSize="20"
Foreground="#CDD6F4"/>
        <TextBlock Text="Chain" FontSize="8"
Foreground="#6C7086" HorizontalAlignment="Center"/>
    </StackPanel>
</Button>
    <Button x:Name="BtnCompare" Tag="Compare"
Click="NavBtn_Click" Width="56" Height="56" Padding="0"
HorizontalAlignment="Center" Background="Transparent" BorderThickness="0"
CornerRadius="12">
        <ToolTipService.ToolTip><ToolTip Content="Compare â€” run the
same prompt on two models side by side"/></ToolTipService.ToolTip>
        <StackPanel Spacing="3" HorizontalAlignment="Center">
            <FontIcon Glyph="#xE8C4;" FontSize="20"
Foreground="#CDD6F4"/>
            <TextBlock Text="Compare" FontSize="8"
Foreground="#6C7086" HorizontalAlignment="Center"/>
        </StackPanel>
    </Button>
    <Button x:Name="BtnPerf" Tag="Performance"
Click="NavBtn_Click" Width="56" Height="56" Padding="0"
HorizontalAlignment="Center" Background="Transparent" BorderThickness="0"
CornerRadius="12">
        <ToolTipService.ToolTip><ToolTip Content="Stats â€” live
performance dashboard, tokens/sec, cost
savings"/></ToolTipService.ToolTip>
        <StackPanel Spacing="3" HorizontalAlignment="Center">
            <FontIcon Glyph="#xE9D9;" FontSize="20"
Foreground="#CDD6F4"/>
            <TextBlock Text="Stats" FontSize="8"
Foreground="#6C7086" HorizontalAlignment="Center"/>
        </StackPanel>
    </Button>
    <Button x:Name="BtnTerminal" Tag="Terminal"
Click="NavBtn_Click" Width="56" Height="56" Padding="0"
HorizontalAlignment="Center" Background="Transparent" BorderThickness="0"
CornerRadius="12">
        <ToolTipService.ToolTip><ToolTip Content="Terminal â€” built-
in command prompt with history"/></ToolTipService.ToolTip>
        <StackPanel Spacing="3" HorizontalAlignment="Center">
            <FontIcon Glyph="#xE756;" FontSize="20"
Foreground="#CDD6F4"/>
            <TextBlock Text="Terminal" FontSize="8"
Foreground="#6C7086" HorizontalAlignment="Center"/>
        </StackPanel>
    </Button>
    <Button x:Name="BtnGuide" Tag="Guide"
Click="NavBtn_Click" Width="56" Height="56" Padding="0"
HorizontalAlignment="Center" Background="Transparent" BorderThickness="0"
CornerRadius="12">
        <ToolTipService.ToolTip><ToolTip Content="Guide â€” help,
keyboard shortcuts, and getting started"/></ToolTipService.ToolTip>
        <StackPanel Spacing="3" HorizontalAlignment="Center">

```

```

        <FontIcon Glyph="&#xE897;" FontSize="20"
Foreground="#CDD6F4"/>
        <TextBlock Text="Guide" FontSize="8"
Foreground="#6C7086" HorizontalAlignment="Center"/>
    </StackPanel>
</Button>
</StackPanel>

<!-- Settings -->
<Button Grid.Row="2" x:Name="BtnSettings" Tag="Settings"
Click="NavBtn_Click"
Width="56" Height="56" Padding="0"
HorizontalAlignment="Center"
Margin="0,0,0,16" Background="Transparent"
BorderThickness="0" CornerRadius="12">
    <ToolTipService.ToolTip><ToolTip Content="Settings â€”
configure Ollama, defaults and theme"/></ToolTipService.ToolTip>
    <StackPanel Spacing="3" HorizontalAlignment="Center">
        <FontIcon Glyph="&#xE713;" FontSize="20"
Foreground="#CDD6F4"/>
        <TextBlock Text="Settings" FontSize="8" Foreground="#6C7086"
HorizontalAlignment="Center"/>
    </StackPanel>
</Button>
</Grid>
</Border>

<Frame x:Name="ContentFrame" Grid.Column="1" Background="#1E1E2E"/>
</Grid>
</Window>
'@ -Encoding UTF8

```

```

Set-Content "$root\MainWindow.xaml.cs" '@
using Microsoft.Extensions.DependencyInjection;
using Microsoft.UI.Xaml;
using Microsoft.UI.Xaml.Controls;
using Microsoft.UI.Xaml.Media;
using OllamaHub.Services;
using OllamaHub.ViewModels;
using OllamaHub.Views;
using Windows.UI;
namespace OllamaHub;
public sealed partial class MainWindow : Window
{
    private readonly MainViewModel _vm;
    private readonly ThemeService _theme;
    private Button? _activeBtn;
    public MainWindow()
    {
        InitializeComponent();
        _vm = App.Services.GetRequiredService<MainViewModel>();
        _theme = App.Services.GetRequiredService<ThemeService>();
        _vm.PropertyChanged += (_, e) => { if (e.PropertyName ==
nameof(MainViewModel.OllamaOnline)) UpdateStatus(); };
    }
}

```



```

        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:models="using:OllamaHub.Models"
        Background="#1E1E2E">
<Grid>
    <Grid.ColumnDefinitions>
        <ColumnDefinition Width="260"/>
        <ColumnDefinition Width="*/>
    </Grid.ColumnDefinitions>

    <!-- SIDEBAR -->
    <Border Grid.Column="0" Background="#181825" BorderBrush="#313244"
BorderThickness="0,0,1,0">
        <Grid>
            <Grid.RowDefinitions>
                <RowDefinition Height="Auto"/>
                <RowDefinition Height="Auto"/>
                <RowDefinition Height="Auto"/>
                <RowDefinition Height="Auto"/>
                <RowDefinition Height="*/>
            </Grid.RowDefinitions>
            <StackPanel Grid.Row="0" Margin="12,14,12,10" Spacing="5">
                <TextBlock Text="MODEL" FontSize="10" FontWeight="SemiBold"
CharacterSpacing="120" Foreground="#6C7086"/>
                <ComboBox x:Name="ModelComboBox" ItemsSource="{x:Bind
ViewModel.AvailableModels}"
                    DisplayMemberPath="Name" SelectedIndex="0"
HorizontalAlignment="Stretch"
                    Background="#313244" BorderBrush="#45475A"
PlaceholderText="Select model...">
                    <ToolTipService.ToolTip><ToolTip Content="Choose which Ollama
model to use for this conversation"/></ToolTipService.ToolTip>
                </ComboBox>
            </StackPanel>
            <!-- Persona badge -->
            <Border Grid.Row="1" x:Name="PersonaBadge" Margin="12,0,12,8"
CornerRadius="8" Padding="10,6"
                    Background="#313244" Visibility="Collapsed">
                <StackPanel Orientation="Horizontal" Spacing="8">
                    <FontIcon x:Name="PersonaIcon" Glyph="⦿" FontSize="14"
Foreground="#7C6AF7"/>
                    <TextBlock x:Name="PersonaName" Text="Persona" FontSize="12"
FontWeight="SemiBold" Foreground="#CDD6F4" VerticalAlignment="Center"/>
                    <Button Click="ClearPersona_Click" Background="Transparent"
BorderThickness="0" Padding="0" Width="16" Height="16">
                        <FontIcon Glyph="✖" FontSize="10"
Foreground="#6C7086"/>
                    </Button>
                </StackPanel>
            </Border>
            <Button Grid.Row="2" Click="NewChat_Click"
HorizontalAlignment="Stretch" Margin="12,0,12,8"
                    CornerRadius="10" Height="36" BorderThickness="0">
                <Button.Background>
                    <LinearGradientBrush StartPoint="0,0" EndPoint="1,0">

```

```

        <GradientStop Color="#7C6AF7" Offset="0"/>
        <GradientStop Color="#60A5FA" Offset="1"/>
    </LinearGradientBrush>
</Button.Background>
<ToolTipService.ToolTip><ToolTip Content="Start a new
conversation (Ctrl+N)"/></ToolTipService.ToolTip>
    <StackPanel Orientation="Horizontal" Spacing="8">
        <FontIcon Glyph="⌂" FontSize="13" Foreground="White"/>
        <TextBlock Text="New Chat" Foreground="White"
FontWeight="SemiBold" FontSize="13"/>
    </StackPanel>
</Button>
    <TextBlock Grid.Row="3" Text="RECENT" FontSize="10"
FontWeight="SemiBold"
        CharacterSpacing="120" Foreground="#6C7086"
Margin="16,4,12,6"/>
    <ListView Grid.Row="4" x:Name="SessionList" ItemsSource="{x:Bind
ViewModel.ChatSessions}"
        SelectionChanged="SessionList_SelectionChanged"
Background="Transparent" Margin="6,0,6,8">
        <ListView.ItemTemplate>
            <DataTemplate x:DataType="models:ChatSession">
                <Border Padding="10,8" CornerRadius="8">
                    <StackPanel Spacing="3">
                        <TextBlock Text="{x:Bind Title}" FontSize="13"
FontWeight="SemiBold" Foreground="#CDD6F4"
TextTrimming="CharacterEllipsis"/>
                        <TextBlock Text="{x:Bind ModelName}" FontSize="11"
Foreground="#6C7086" TextTrimming="CharacterEllipsis"/>
                    </StackPanel>
                </Border>
            </DataTemplate>
        </ListView.ItemTemplate>
        <ListView.ItemContainerStyle>
            <Style TargetType="ListViewItem">
                <Setter Property="HorizontalContentAlignment"
Value="Stretch"/>
                <Setter Property="CornerRadius" Value="8"/>
                <Setter Property="Margin" Value="0,1"/>
                <Setter Property="Padding" Value="0"/>
            </Style>
        </ListView.ItemContainerStyle>
    </ListView>
</Grid>
</Border>

<!-- MAIN AREA -->
<Grid Grid.Column="1">
    <Grid.RowDefinitions>
        <RowDefinition Height="Auto"/>
        <RowDefinition Height="*" />
        <RowDefinition Height="Auto"/>
        <RowDefinition Height="Auto"/>
    </Grid.RowDefinitions>

```



```

<!-- Top bar -->
<Border Grid.Row="0" Background="#181825" BorderBrush="#313244"
BorderThickness="0,0,0,1" Padding="20,10">
    <Grid>
        <Grid.ColumnDefinitions>
            <ColumnDefinition Width="*"/>
            <ColumnDefinition Width="Auto"/>
        </Grid.ColumnDefinitions>
        <StackPanel Grid.Column="0" Spacing="4">
            <StackPanel Orientation="Horizontal" Spacing="10">
                <TextBlock Text="Context" FontSize="11"
Foreground="#6C7086" VerticalAlignment="Center"/>
                <TextBlock Text="{x:Bind ViewModel.ContextTokens,
Mode=OneWay}" FontSize="11" Foreground="#9399B2"
VerticalAlignment="Center"/>
                <TextBlock Text="/" FontSize="11" Foreground="#45475A"
VerticalAlignment="Center"/>
                <TextBlock Text="{x:Bind ViewModel.MaxContextTokens,
Mode=OneWay}" FontSize="11" Foreground="#6C7086"
VerticalAlignment="Center"/>
                <TextBlock Text="tokens" FontSize="11" Foreground="#6C7086"
VerticalAlignment="Center"/>
                <TextBlock Text="{x:Bind ViewModel.StatusText,
Mode=OneWay}" FontSize="11" Foreground="#6C7086"
VerticalAlignment="Center" Margin="12,0,0,0"/>
            </StackPanel>
            <Border Background="#313244" CornerRadius="3" Height="4"
Width="200" HorizontalAlignment="Left">
                <Border x:Name="ContextBar" CornerRadius="3" Height="4"
HorizontalAlignment="Left" Width="0">
                    <Border.Background><SolidColorBrush
x:Name="ContextBarBrush" Color="#4ADE80"/></Border.Background>
                </Border>
            </Border>
        </StackPanel>
        <DropDownButton Grid.Column="1" CornerRadius="8"
Background="#313244" BorderBrush="#45475A" Foreground="#CDD6F4"
FontSize="12">
            <ToolTipService.ToolTip><ToolTip Content="Share or export
this conversation"/></ToolTipService.ToolTip>
            <StackPanel Orientation="Horizontal" Spacing="6">
                <FontIcon Glyph="&#xE72D;" FontSize="12"
Foreground="#7C6AF7"/>
                <TextBlock Text="Share"/>
            </StackPanel>
            <DropDownButton.Flyout>
                <MenuFlyout>
                    <MenuFlyoutItem Text="Copy to clipboard"
Click="CopyClipboard_Click"><MenuFlyoutItem.Icon><FontIcon
Glyph="&#xE8C8;" /></MenuFlyoutItem.Icon></MenuFlyoutItem>
                    <MenuFlyoutSeparator/>

```

```

        <MenuFlyoutItem Text="Save as Markdown"
Click="ExportMd_Click"><MenuFlyoutItem.Icon><FontIcon
Glyph="#xE8A5;" /></MenuFlyoutItem.Icon></MenuFlyoutItem>
        <MenuFlyoutItem Text="Save as HTML"
Click="ExportHtml_Click"><MenuFlyoutItem.Icon><FontIcon
Glyph="#xF6FA;" /></MenuFlyoutItem.Icon></MenuFlyoutItem>
        <MenuFlyoutItem Text="Save as Text"
Click="ExportTxt_Click"><MenuFlyoutItem.Icon><FontIcon
Glyph="#xE8A4;" /></MenuFlyoutItem.Icon></MenuFlyoutItem>
    </MenuFlyout>
</DropDownButton.Flyout>
</DropDownButton>
</Grid>
</Border>

<!-- Messages -->
<ScrollViewer Grid.Row="1" x:Name="MessagesScroll"
VerticalScrollBarVisibility="Auto" Padding="28,20,28,12">
    <StackPanel Spacing="20">
        <StackPanel x:Name="EmptyState" HorizontalAlignment="Center"
Margin="0,60,0,0" Spacing="16">
            <Border Width="80" Height="80" CornerRadius="24"
HorizontalAlignment="Center">
                <Border.Background>
                    <LinearGradientBrush StartPoint="0,0" EndPoint="1,1">
                        <GradientStop Color="#7C6AF7" Offset="0"/>
                        <GradientStop Color="#60A5FA" Offset="1"/>
                    </LinearGradientBrush>
                </Border.Background>
                <FontIcon Glyph="#xE99A;" FontSize="36" Foreground="White"
HorizontalAlignment="Center" VerticalAlignment="Center"/>
            </Border>
            <TextBlock Text="OllamaHub" FontSize="32" FontWeight="Bold"
Foreground="#CDD6F4" HorizontalAlignment="Center"/>
            <TextBlock Text="Type / for prompt templates · Select a
persona · Or just start chatting"
                FontSize="13" Foreground="#6C7086"
HorizontalAlignment="Center"/>
            <StackPanel Orientation="Horizontal" Spacing="10"
HorizontalAlignment="Center" Margin="0,8,0,0">
                <Border Background="#313244" BorderBrush="#45475A"
BorderThickness="1" CornerRadius="10" Padding="14,10">
                    <StackPanel Spacing="6" HorizontalAlignment="Center">
                        <FontIcon Glyph="#xE943;" FontSize="18"
Foreground="#7C6AF7" HorizontalAlignment="Center"/>
                        <TextBlock Text="CoPilot" FontSize="11"
Foreground="#6C7086" HorizontalAlignment="Center"/>
                    </StackPanel>
                </Border>
                <Border Background="#313244" BorderBrush="#45475A"
BorderThickness="1" CornerRadius="10" Padding="14,10">
                    <StackPanel Spacing="6" HorizontalAlignment="Center">
                        <FontIcon Glyph="#xE8EF;" FontSize="18"
Foreground="#60A5FA" HorizontalAlignment="Center"/>

```

```

        <TextBlock Text="Chain" FontSize="11"
Foreground="#6C7086" HorizontalAlignment="Center"/>
    </StackPanel>
</Border>
<Border Background="#313244" BorderBrush="#45475A"
BorderThickness="1" CornerRadius="10" Padding="14,10">
    <StackPanel Spacing="6" HorizontalAlignment="Center">
        <FontIcon Glyph="⌘" FontSize="18"
Foreground="#F472B6" HorizontalAlignment="Center"/>
        <TextBlock Text="Personas" FontSize="11"
Foreground="#6C7086" HorizontalAlignment="Center"/>
    </StackPanel>
</Border>
</StackPanel>
</StackPanel>

<ItemsControl x:Name="MessagesList" ItemsSource="{x:Bind
ViewModel.Messages, Mode=OneWay}">
    <ItemsControl.ItemTemplate>
        <DataTemplate x:DataType="models:ChatMessage">
            <Grid Margin="0,6">
                <Border Visibility="{x:Bind IsUser,
Converter={StaticResource BoolToVisibilityConverter}}"
                    HorizontalAlignment="Right" MaxWidth="620"
                    CornerRadius="18,18,4,18" Padding="16,12"
Margin="100,0,0,0">
                    <Border.Background>
                        <LinearGradientBrush StartPoint="0,0"
EndPoint="1,1">
                            <GradientStop Color="#7C6AF7" Offset="0"/>
                            <GradientStop Color="#60A5FA" Offset="1"/>
                        </LinearGradientBrush>
                    </Border.Background>
                    <TextBlock Text="{x:Bind Content}" Foreground="White"
TextWrapping="Wrap"
                        FontSize="14"
IsTextSelectionEnabled="True" LineHeight="22"/>
                </Border>
                <Border Visibility="{x:Bind IsAssistant,
Converter={StaticResource BoolToVisibilityConverter}}"
                    HorizontalAlignment="Left" MaxWidth="720"
                    Background="#313244" BorderBrush="#45475A"
BorderThickness="1"
                    CornerRadius="18,18,18,4" Padding="18,14"
Margin="0,0,100,0">
                    <StackPanel Spacing="10">
                        <StackPanel Orientation="Horizontal" Spacing="6">
                            <Border Background="#1E1E2E" CornerRadius="6"
Padding="6,3">
                                <StackPanel Orientation="Horizontal"
Spacing="5">
                                    <FontIcon Glyph="⌘" FontSize="10"
Foreground="#7C6AF7"/>

```

```

        <TextBlock Text="{x:Bind ModelName}"
FontSize="11" FontWeight="SemiBold" Foreground="#7C6AF7"/>
        </StackPanel>
    </Border>
</StackPanel>
    <TextBlock Text="{x:Bind Content}"
TextWrapping="Wrap" FontSize="14"
        Foreground="#CDD6F4"
IsTextSelectionEnabled="True" LineHeight="22"/>
    <ProgressRing IsActive="{x:Bind IsStreaming}"
Width="18" Height="18" Foreground="#7C6AF7"
        Visibility="{x:Bind IsStreaming,
Converter={StaticResource BoolToVisibilityConverter}}"/>
    </StackPanel>
</Border>
</Grid>
</DataTemplate>
</ItemsControl.ItemTemplate>
</ItemsControl>
</StackPanel>
</ScrollViewer>

<!-- Slash popup -->
<Border Grid.Row="2" x:Name="SlashPopup" Visibility="Collapsed"
    Background="#313244" BorderBrush="#7C6AF7"
BorderThickness="1"
    CornerRadius="12" Margin="20,0,20,4" Padding="8">
    <ListView x:Name="SlashList" MaxHeight="220"
Background="Transparent" SelectionChanged="SlashList_SelectionChanged">
        <ListView.ItemTemplate>
            <DataTemplate>
                <Grid ColumnSpacing="10" Padding="6,4">
                    <Grid.ColumnDefinitions>
                        <ColumnDefinition Width="110"/>
                        <ColumnDefinition Width="*"/>
                        <ColumnDefinition Width="*"/>
                    </Grid.ColumnDefinitions>
                    <TextBlock Grid.Column="0" Text="{Binding Trigger}"
FontSize="13" FontWeight="SemiBold" Foreground="#7C6AF7"
FontFamily="Cascadia Code,Consolas,monospace"/>
                    <TextBlock Grid.Column="1" Text="{Binding Name}"
FontSize="13" Foreground="#CDD6F4"/>
                    <TextBlock Grid.Column="2" Text="{Binding Description}"
FontSize="11" Foreground="#6C7086" TextTrimming="CharacterEllipsis"/>
                </Grid>
            </DataTemplate>
        </ListView.ItemTemplate>
        <ListView.ItemContainerStyle>
            <Style TargetType="ListViewItem">
                <Setter Property="HorizontalContentAlignment"
Value="Stretch"/>
                <Setter Property="CornerRadius" Value="6"/>
                <Setter Property="Padding" Value="0"/>
            </Style>
        </ListView.ItemContainerStyle>
    </ListView>
</Border>

```

```

        </ListView.ItemContainerStyle>
    </ListView>
</Border>

<!-- Input bar -->
<Border Grid.Row="3" Background="#181825" BorderBrush="#313244"
BorderThickness="0,1,0,0" Padding="20,12,20,18">
    <Grid ColumnSpacing="10">
        <Grid.ColumnDefinitions>
            <ColumnDefinition Width="Auto"/>
            <ColumnDefinition Width="*/>
            <ColumnDefinition Width="Auto"/>
            <ColumnDefinition Width="Auto"/>
        </Grid.ColumnDefinitions>
        <Button Grid.Column="0" x:Name="VoiceBtn"
Click="VoiceBtn_Click"
            Width="44" Height="44" CornerRadius="12"
Background="#313244" BorderBrush="#45475A" BorderThickness="1">
            <ToolTipService.ToolTip><ToolTip Content="Voice input â€”
click to start microphone (purple =
listening)"/></ToolTipService.ToolTip>
            <FontIcon x:Name="VoiceIcon" Glyph="&#xE720;" FontSize="18"
Foreground="#6C7086"/>
        </Button>
        <Border Grid.Column="1" Background="#313244"
BorderBrush="#45475A" BorderThickness="1" CornerRadius="14">
            <TextBox x:Name="InputBox"
                Text="{x:Bind ViewModel.InputText, Mode=TwoWay,
UpdateSourceTrigger=PropertyChanged}"
                PlaceholderText="Message OllamaHub... (/ for
commands, Enter to send)"
                AcceptsReturn="False" MaxHeight="120"
TextWrapping="Wrap"
                BorderThickness="0" Background="Transparent"
Foreground="#CDD6F4"
                Padding="16,12" FontSize="14"
                KeyDown="InputBox_KeyDown"
TextChanged="InputBox_TextChanged" VerticalAlignment="Center"/>
        </Border>
        <Button Grid.Column="2" x:Name="TtsBtn" Click="TtsBtn_Click"
            Width="44" Height="44" CornerRadius="12"
Background="#313244" BorderBrush="#45475A" BorderThickness="1">
            <ToolTipService.ToolTip><ToolTip Content="Text-to-speech â€”
model reads responses aloud (purple = on)"/></ToolTipService.ToolTip>
            <FontIcon x:Name="TtsIcon" Glyph="&#xE995;" FontSize="18"
Foreground="#6C7086"/>
        </Button>
        <Button Grid.Column="3" x:Name="SendButton"
Click="SendButton_Click"
            Width="50" Height="50" CornerRadius="14"
BorderThickness="0">
            <Button.Background>
                <LinearGradientBrush StartPoint="0,0" EndPoint="1,1">
                    <GradientStop Color="#7C6AF7" Offset="0"/>

```

```

        <GradientStop Color="#60A5FA" Offset="1"/>
    </LinearGradientBrush>
</Button.Background>
<ToolTipService.ToolTip><ToolTip Content="Send message
(Enter) â€œ click again to stop generation"/></ToolTipService.ToolTip>
    <FontIcon x:Name="SendIcon" Glyph="&#xE724;" FontSize="18"
Foreground="White"/>
</Button>
</Grid>
</Border>
</Grid>
</Grid>
</Page>
'@ -Encoding UTF8
Write-Ok "Views\ChatPage.xaml"

```

```

Write-Step "Writing Views\ChatPage.xaml.cs"
Set-Content "$root\Views\ChatPage.xaml.cs" '@
using Microsoft.Extensions.DependencyInjection;
using Microsoft.UI.Xaml;
using Microsoft.UI.Xaml.Controls;
using Microsoft.UI.Xaml.Input;
using Microsoft.UI.Xaml.Media;
using OllamaHub.Models;
using OllamaHub.Services;
using OllamaHub.ViewModels;
using Windows.System;
using Windows.UI;
namespace OllamaHub.Views;
public sealed partial class ChatPage : Page
{
    public ChatViewModel ViewModel { get; }
    private readonly ShareService _share;
    private readonly VoiceService _voice;
    private readonly SlashCommandService _slash;
    private bool _ttsEnabled, _slashVisible;
    public static string? PendingPrompt { get; set; }
    public static Persona? PendingPersona { get; set; }
    public ChatPage()
    {
        ViewModel = App.Services.GetRequiredService<ChatViewModel>();
        _share = App.Services.GetRequiredService<ShareService>();
        _voice = App.Services.GetRequiredService<VoiceService>();
        _slash =
App.Services.GetRequiredService<SlashCommandService>();
        InitializeComponent();
        _voice.SpeechRecognized += (_, text) =>
DispatcherQueue.TryEnqueue(() => ViewModel.InputText +=
(ViewModel.InputText.Length > 0 ? " " : "") + text);
        _voice.ListeningStarted += (_, _) =>
DispatcherQueue.TryEnqueue(() => { VoiceIcon.Foreground=new
SolidColorBrush(Color.FromArgb(255,124,106,247)); VoiceBtn.Background=new
SolidColorBrush(Color.FromArgb(40,124,106,247)); });

```

```

        _voice.ListeningStopped += (_, _) =>
DispatcherQueue.TryEnqueue(() => { VoiceIcon.Foreground=new
SolidColorBrush(Color.FromArgb(255,108,112,134)); VoiceBtn.Background=new
SolidColorBrush(Color.FromArgb(255,49,50,68)); });
        _voice.ErrorOccurred += async (_, msg) =>
{
    await DispatcherQueue.TryEnqueue(async () =>
    {
        var dlg = new ContentDialog { Title="Voice error",
Content=msg, CloseButtonText="OK", XamlRoot=XamlRoot };
        await dlg.ShowAsync();
    });
};
        ViewModel.PropertyChanged += (_, e) => { if (e.PropertyName ==
nameof(ViewModel.ContextPercent)) UpdateContextBar(); };
        ViewModel.Messages.CollectionChanged += (_, _) =>
ScrollToBottom();
        Loaded += (_, _) =>
{
    if (PendingPersona != null) { ApplyPersona(PendingPersona);
PendingPersona = null; }
    if (!string.IsNullOrEmpty(PendingPrompt)) {
ViewModel.InputText = PendingPrompt; PendingPrompt = null; }
    UpdateContextBar(); InputBox.Focus(FocusState.Programmatic);
    ModelComboBox.SelectionChanged += (s, e) => { if
(ModelComboBox.SelectedItem is OllamaModelInfo m) ViewModel.SelectedModel
= m.Name; };
};
    }
    private void ApplyPersona(Persona p)
    {
        ViewModel.ActivPersona = p;
        PersonaBadge.Visibility = Visibility.Visible;
        PersonaName.Text = p.Name;
        PersonaIcon.Foreground = new SolidColorBrush(Color.FromArgb(255,
Convert.ToByte(p.AvatarColor[1..3], 16),
Convert.ToByte(p.AvatarColor[3..5], 16),
Convert.ToByte(p.AvatarColor[5..7], 16)));
    }
    private void ClearPersona_Click(object s, RoutedEventArgs e) {
ViewModel.ActivPersona = null; PersonaBadge.Visibility =
Visibility.Collapsed; }
    private void UpdateContextBar()
    {
        DispatcherQueue.TryEnqueue(() =>
        {
            ContextBar.Width = 200.0 * ViewModel.ContextPercent / 100.0;
            ContextBarBrush.Color = ViewModel.ContextColor switch
            {
                "#FB923C" => Color.FromArgb(255,251,146,60),
                "#F87171" => Color.FromArgb(255,248,113,113),
                _ => Color.FromArgb(255,74,222,128)
            };
        });
    }
};

```

```

    }
    private void ScrollToBottom() { DispatcherQueue.TryEnqueue(() => {
MessagesScroll.UpdateLayout();
MessagesScroll.ScrollToVerticalOffset(MessagesScroll.ExtentHeight); }); }
    private void NewChat_Click(object s, RoutedEventArgs e) =>
ViewModel.NewChatCommand.Execute(null);
    private void SessionList_SelectionChanged(object s,
SelectionChangedEventArgs e) { if (SessionList.SelectedItem is
ChatSession session) ViewModel.OpenSessionCommand.Execute(session); }
    private void InputBox_KeyDown(object sender, KeyRoutedEventArgs e)
    {
        if (e.Key == VirtualKey.Enter)
        {
            if (_slashVisible && SlashList.SelectedItem != null) {
ApplySlashCommand(SlashList.SelectedItem); e.Handled=true; return; }
            var ctrl =
Microsoft.UI.Input.InputKeyboardSource.GetKeyStateForCurrentThread(Virtua
lKey.Control);
            if (!ctrl.HasFlag(Windows.UI.Core.CoreVirtualKeyStates.Down))
{ HideSlash(); SendButton_Click(sender, e); e.Handled=true; }
        }
        else if (e.Key==VirtualKey.Escape) HideSlash();
        else if (e.Key==VirtualKey.Up && _slashVisible) {
if(SlashList.SelectedIndex>0) SlashList.SelectedIndex--; e.Handled=true;
}
        else if (e.Key==VirtualKey.Down && _slashVisible) {
if(SlashList.SelectedIndex<SlashList.Items.Count-1)
SlashList.SelectedIndex++; e.Handled=true; }
    }
    private void InputBox_TextChanged(object sender, TextChangedEventArgs
e)
    {
        var text = ViewModel.InputText;
        if (text.StartsWith("/")) { var
results=_slash.Search(text).ToList();
if(results.Any()){SlashList.ItemsSource=results;SlashList.SelectedIndex=0
;SlashPopup.Visibility=Visibility.Visible;_slashVisible=true;return;} }
        HideSlash();
    }
    private void SlashList_SelectionChanged(object s,
SelectionChangedEventArgs e) { }
    private void ApplySlashCommand(object item) { if(item is SlashCommand
cmd){ViewModel.InputText=cmd.Content;HideSlash();InputBox.Focus(FocusStat
e.Programmatic);} }
    private void HideSlash() {
SlashPopup.Visibility=Visibility.Collapsed; _slashVisible=false; }
    private void SendButton_Click(object sender, RoutedEventArgs e)
    {
        if (ViewModel.IsGenerating) { _voice.StopSpeaking();
ViewModel.StopGenerationCommand.Execute(null); SendIcon.Glyph="\uE724"; }
        else { ViewModel.SendMessageCommand.Execute(null);
SendIcon.Glyph="\uE71A"; ViewModel.PropertyChanged+=OnGenChanged; }
    }

```



```

</Grid.ColumnDefinitions>

<!-- LEFT: Code Input + Actions -->
<Grid Grid.Column="0" Margin="0,0,1,0">
  <Grid.RowDefinitions>
    <RowDefinition Height="Auto"/>
    <RowDefinition Height="Auto"/>
    <RowDefinition Height="*/>
    <RowDefinition Height="Auto"/>
  </Grid.RowDefinitions>

  <!-- Header -->
  <Border Grid.Row="0" Background="#181825" BorderBrush="#313244"
BorderThickness="0,0,0,1" Padding="20,14">
    <Grid>
      <Grid.ColumnDefinitions>
        <ColumnDefinition Width="*/>
        <ColumnDefinition Width="Auto"/>
      </Grid.ColumnDefinitions>
      <StackPanel Grid.Column="0" Orientation="Horizontal"
Spacing="12">
        <Border Background="#313244" CornerRadius="8" Padding="8">
          <FontIcon Glyph="&#xE943;" FontSize="18"
Foreground="#7C6AF7"/>
        </Border>
        <StackPanel VerticalAlignment="Center" Spacing="1">
          <TextBlock Text="OllamaHub CoPilot" FontSize="16"
FontWeight="SemiBold" Foreground="#CDD6F4"/>
          <TextBlock Text="AI code assistant powered by local Ollama"
FontSize="11" Foreground="#6C7086"/>
        </StackPanel>
      </StackPanel>
      <StackPanel Grid.Column="1" Orientation="Horizontal"
Spacing="10">
        <TextBlock Text="MODEL" FontSize="10" FontWeight="SemiBold"
CharacterSpacing="120" Foreground="#6C7086" VerticalAlignment="Center"/>
        <ComboBox x:Name="ModelBox" Width="180" ItemsSource="{x:Bind
ViewModel.AvailableModels}"
DisplayMemberPath="Name" Background="#313244"
BorderBrush="#45475A"
PlaceholderText="Select model...">
          <ToolTipService.ToolTip><ToolTip Content="Choose the model
for code assistance. Code-specific models (codestral, deepseek-coder)
work best."/></ToolTipService.ToolTip>
        </ComboBox>
      </StackPanel>
    </Grid>
  </Border>

  <!-- Mode buttons -->
  <Border Grid.Row="1" Background="#181825" BorderBrush="#313244"
BorderThickness="0,0,0,1" Padding="16,10">
    <StackPanel Orientation="Horizontal" Spacing="8">

```

```

        <TextBlock Text="MODE:" FontSize="11" FontWeight="SemiBold"
CharacterSpacing="80" Foreground="#6C7086" VerticalAlignment="Center"/>
        <Button x:Name="BtnModeChat" Content="Chat"
Click="Mode_Click" Tag="Chat" CornerRadius="8" Padding="14,6"
FontSize="12" Background="#7C6AF7" BorderThickness="0"
Foreground="White"/>
        <Button x:Name="BtnModeExplain" Content="Explain"
Click="Mode_Click" Tag="Explain" CornerRadius="8" Padding="14,6"
FontSize="12" Background="#313244" BorderBrush="#45475A"/>
        <Button x:Name="BtnModeReview" Content="Review"
Click="Mode_Click" Tag="Review" CornerRadius="8" Padding="14,6"
FontSize="12" Background="#313244" BorderBrush="#45475A"/>
        <Button x:Name="BtnModeFix" Content="Fix"
Click="Mode_Click" Tag="Fix" CornerRadius="8" Padding="14,6"
FontSize="12" Background="#313244" BorderBrush="#45475A"/>
        <Button x:Name="BtnModeTests" Content="Tests"
Click="Mode_Click" Tag="Tests" CornerRadius="8" Padding="14,6"
FontSize="12" Background="#313244" BorderBrush="#45475A"/>
        <Button x:Name="BtnModeDocs" Content="Docs"
Click="Mode_Click" Tag="Docs" CornerRadius="8" Padding="14,6"
FontSize="12" Background="#313244" BorderBrush="#45475A"/>
        <Button x:Name="BtnModeRefactor" Content="Refactor"
Click="Mode_Click" Tag="Refactor" CornerRadius="8" Padding="14,6"
FontSize="12" Background="#313244" BorderBrush="#45475A"/>
    </StackPanel>
</Border>

```

```

<!-- Code / chat input -->
<Border Grid.Row="2" Background="#1E1E2E" Margin="0">
    <Grid>
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="*/>
        </Grid.RowDefinitions>
        <!-- Code editor area -->
        <Border Grid.Row="0" x:Name="CodeInputArea"
Background="#181825" BorderBrush="#313244" BorderThickness="0,0,0,1"
Padding="0">
            <Grid>
                <Grid.RowDefinitions>
                    <RowDefinition Height="Auto"/>
                    <RowDefinition Height="200"/>
                </Grid.RowDefinitions>
                <Border Grid.Row="0" Padding="16,8" Background="#13131E">
                    <StackPanel Orientation="Horizontal" Spacing="8">
                        <FontIcon Glyph="✂" FontSize="12"
Foreground="#6C7086"/>
                        <TextBlock Text="Paste your code here" FontSize="12"
Foreground="#6C7086"/>
                    </StackPanel>
                </Border>
                <TextBox Grid.Row="1" x:Name="CodeBox"
Text="{x:Bind ViewModel.CodeInput, Mode=TwoWay,
UpdateSourceTrigger=PropertyChanged}"

```

```

        PlaceholderText="// Paste your code here for
Explain, Review, Fix, Tests, Docs, or Refactor modes..."
        AcceptsReturn="True" TextWrapping="Wrap"
        BorderThickness="0" Background="Transparent"
        Foreground="#CDD6F4" Padding="16,12"
        FontFamily="Cascadia Code, Consolas, monospace"
FontSize="13"/>
    </Grid>
</Border>
<!-- Chat message area (Chat mode) -->
<ScrollViewer Grid.Row="1" x:Name="ChatScroll"
VerticalScrollBarVisibility="Auto" Padding="16,12">
    <ItemsControl ItemsSource="{x:Bind ViewModel.Messages,
Mode=OneWay}">
        <ItemsControl.ItemTemplate>
            <DataTemplate x:DataType="vm:CopilotMessage">
                <Grid Margin="0,6">
                    <Border Visibility="{x:Bind IsUser,
Converter={StaticResource BoolToVisibilityConverter}}"
                        HorizontalAlignment="Right" MaxWidth="500"
CornerRadius="14,14,4,14" Padding="14,10" Margin="80,0,0,0">
                        <Border.Background>
                            <LinearGradientBrush StartPoint="0,0"
EndPoint="1,1">
                                <GradientStop Color="#7C6AF7" Offset="0"/>
                                <GradientStop Color="#60A5FA" Offset="1"/>
                            </LinearGradientBrush>
                        </Border.Background>
                        <TextBlock Text="{x:Bind Content}"
Foreground="White" TextWrapping="Wrap" FontSize="13"
IsTextSelectionEnabled="True"/>
                    </Border>
                    <Border Visibility="{x:Bind IsAssistant,
Converter={StaticResource BoolToVisibilityConverter}}"
                        HorizontalAlignment="Left" MaxWidth="580"
Background="#313244" BorderBrush="#45475A" BorderThickness="1"
                        CornerRadius="14,14,14,4" Padding="14,12"
Margin="0,0,80,0">
                        <StackPanel Spacing="8">
                            <StackPanel Orientation="Horizontal" Spacing="6">
                                <FontIcon Glyph="⌘" FontSize="11"
Foreground="#7C6AF7"/>
                                <TextBlock Text="{x:Bind Model}" FontSize="11"
FontWeight="SemiBold" Foreground="#7C6AF7"/>
                            </StackPanel>
                            <TextBlock Text="{x:Bind Content}"
TextWrapping="Wrap" FontSize="13" Foreground="#CDD6F4"
                                IsTextSelectionEnabled="True"
FontFamily="Cascadia Code, Consolas, monospace" LineHeight="20"/>
                        </StackPanel>
                    </Border>
                </Grid>
            </DataTemplate>
        </ItemsControl.ItemTemplate>
    </ItemsControl>
</ScrollViewer>
</Grid>
</DataTemplate>
</ItemsControl.ItemTemplate>

```

```

        </ItemsControl>
    </ScrollViewer>
</Grid>
</Border>

<!-- Input bar -->
<Border Grid.Row="3" Background="#181825" BorderBrush="#313244"
BorderThickness="0,1,0,0" Padding="16,12">
    <Grid ColumnSpacing="10">
        <Grid.ColumnDefinitions>
            <ColumnDefinition Width="*" />
            <ColumnDefinition Width="Auto" />
            <ColumnDefinition Width="Auto" />
        </Grid.ColumnDefinitions>
        <Border Grid.Column="0" Background="#313244"
BorderBrush="#45475A" BorderThickness="1" CornerRadius="12">
            <TextBox x:Name="CopilotInput"
                Text="{x:Bind ViewModel.InputText, Mode=TwoWay,
UpdateSourceTrigger=PropertyChanged}"
                PlaceholderText="Ask CoPilot... or describe the
issue to fix"
                BorderThickness="0" Background="Transparent"
                Foreground="#CDD6F4"
                Padding="14,11" FontSize="13"
                KeyDown="CopilotInput_KeyDown" />
            </Border>
            <Button Grid.Column="1" Click="Run_Click"
                IsEnabled="{x:Bind ViewModel.IsGenerating, Mode=OneWay,
Converter={StaticResource InverseBoolConverter}}"
                Width="48" Height="44" CornerRadius="12"
BorderThickness="0">
                <Button.Background>
                    <LinearGradientBrush StartPoint="0,0" EndPoint="1,1">
                        <GradientStop Color="#7C6AF7" Offset="0" />
                        <GradientStop Color="#60A5FA" Offset="1" />
                    </LinearGradientBrush>
                </Button.Background>
                <FontIcon Glyph="⌂" FontSize="16" Foreground="White" />
            </Button>
            <Button Grid.Column="2" Click="Clear_Click" Width="44"
Height="44" CornerRadius="12"
                Background="#313244" BorderBrush="#45475A"
ToolTipService.ToolTip="Clear">
                <FontIcon Glyph="✖" FontSize="15"
                Foreground="#F87171" />
            </Button>
        </Grid>
    </Border>
</Grid>

<!-- RIGHT: Output panel -->
<Border Grid.Column="1" Background="#181825" BorderBrush="#313244"
BorderThickness="1,0,0,0">
    <Grid>

```

```

        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="*/>
            <RowDefinition Height="Auto"/>
        </Grid.RowDefinitions>
        <Border Grid.Row="0" Padding="16,12" BorderBrush="#313244"
BorderThickness="0,0,0,1">
            <Grid>
                <Grid.ColumnDefinitions><ColumnDefinition
Width="*/><ColumnDefinition Width="Auto"/></Grid.ColumnDefinitions>
                <StackPanel Grid.Column="0" Orientation="Horizontal"
Spacing="8">
                    <TextBlock Text="OUTPUT" FontSize="10"
FontWeight="SemiBold" CharacterSpacing="120" Foreground="#6C7086"
VerticalAlignment="Center"/>
                    <ProgressRing IsActive="{x:Bind ViewModel.IsGenerating,
Mode=OneWay}" Width="14" Height="14" Foreground="#7C6AF7"/>
                    <TextBlock Text="{x:Bind ViewModel.StatusText,
Mode=OneWay}" FontSize="11" Foreground="#6C7086"
VerticalAlignment="Center"/>
                </StackPanel>
                <Button Grid.Column="1" Click="CopyOutput_Click"
Background="Transparent" BorderThickness="0"
                    ToolTipService.ToolTip="Copy output to clipboard"
Padding="6">
                    <StackPanel Orientation="Horizontal" Spacing="4">
                        <FontIcon Glyph="✂" FontSize="13"
Foreground="#6C7086"/>
                        <TextBlock Text="Copy" FontSize="11"
Foreground="#6C7086"/>
                    </StackPanel>
                </Button>
            </Grid>
        </Border>
        <ScrollViewer Grid.Row="1" VerticalScrollBarVisibility="Auto"
Padding="16,12">
            <TextBlock Text="{x:Bind ViewModel.OutputText, Mode=OneWay}"
TextWrapping="Wrap" FontSize="13"
Foreground="#CDD6F4"
                IsTextSelectionEnabled="True" LineHeight="20"
                FontFamily="Cascadia Code,Consolas,monospace"/>
        </ScrollViewer>
        <Border Grid.Row="2" Padding="16,10" BorderBrush="#313244"
BorderThickness="0,1,0,0" Background="#13131E">
            <TextBlock FontSize="11" Foreground="#45475A"
TextWrapping="Wrap">
                <Run Text="Tip: "/>
                <Run Text="Select a mode above, paste your code on the left,
and click Run. In Chat mode, just type your question."
Foreground="#6C7086"/>
            </TextBlock>
        </Border>
    </Grid>
</Border>

```



```

        <TextBlock Text="Personas" FontSize="16"
FontWeight="SemiBold" Foreground="#CDD6F4" VerticalAlignment="Center"/>
    </StackPanel>
</Border>
    <Button Grid.Row="1" Click="NewPersona_Click"
HorizontalAlignment="Stretch" Margin="12,0,12,10"
        CornerRadius="10" Height="36" BorderThickness="0">
        <Button.Background>
            <LinearGradientBrush StartPoint="0,0" EndPoint="1,0">
                <GradientStop Color="#7C6AF7" Offset="0"/>
                <GradientStop Color="#F472B6" Offset="1"/>
            </LinearGradientBrush>
        </Button.Background>
        <StackPanel Orientation="Horizontal" Spacing="8">
            <FontIcon Glyph="⌕" FontSize="13" Foreground="White"/>
            <TextBlock Text="New Persona" Foreground="White"
FontWeight="SemiBold" FontSize="13"/>
        </StackPanel>
    </Button>
    <ListView Grid.Row="2" x:Name="PersonaList" ItemsSource="{x:Bind
ViewModel.Personas}"
        SelectionChanged="PersonaList_SelectionChanged"
Background="Transparent" Margin="6,0,6,8">
        <ListView.ItemTemplate>
            <DataTemplate x:DataType="models:Persona">
                <Border Padding="10,10" CornerRadius="10">
                    <Grid ColumnSpacing="10">
                        <Grid.ColumnDefinitions>
                            <ColumnDefinition Width="Auto"/>
                            <ColumnDefinition Width="*" />
                        </Grid.ColumnDefinitions>
                        <Border Grid.Column="0" Width="38" Height="38"
CornerRadius="10"
                            Background="{x:Bind AvatarColor}">
                            <FontIcon Glyph="{x:Bind Avatar}" FontSize="18"
Foreground="White"
                                HorizontalAlignment="Center"
VerticalAlignment="Center"/>
                        </Border>
                        <StackPanel Grid.Column="1" Spacing="2"
VerticalAlignment="Center">
                            <TextBlock Text="{x:Bind Name}" FontSize="13"
FontWeight="SemiBold" Foreground="#CDD6F4"
TextTrimming="CharacterEllipsis"/>
                            <TextBlock Text="{x:Bind Description}" FontSize="11"
Foreground="#6C7086" TextTrimming="CharacterEllipsis"/>
                        </StackPanel>
                    </Grid>
                </Border>
            </DataTemplate>
        </ListView.ItemTemplate>
    </ListView.ItemContainerStyle>
    <Style TargetType="ListViewItem">

```

```

        <Setter Property="HorizontalContentAlignment"
Value="Stretch"/>
        <Setter Property="CornerRadius" Value="10"/>
        <Setter Property="Margin" Value="0,2"/>
        <Setter Property="Padding" Value="0"/>
    </Style>
</ListView.ItemContainerStyle>
</ListView>
</Grid>
</Border>

<!-- RIGHT: Editor -->
<Grid Grid.Column="1" Padding="28">
    <Grid.RowDefinitions>
        <RowDefinition Height="Auto"/>
        <RowDefinition Height="Auto"/>
        <RowDefinition Height="Auto"/>
        <RowDefinition Height="*/>
        <RowDefinition Height="Auto"/>
    </Grid.RowDefinitions>
    <TextBlock Grid.Row="0" Text="Edit Persona" FontSize="22"
FontWeight="Bold" Foreground="#CDD6F4" Margin="0,0,0,20"/>
    <StackPanel Grid.Row="1" Orientation="Horizontal" Spacing="16"
Margin="0,0,0,14">
        <StackPanel Spacing="6" Width="200">
            <TextBlock Text="NAME" FontSize="10" FontWeight="SemiBold"
CharacterSpacing="120" Foreground="#6C7086"/>
            <Border Background="#313244" BorderBrush="#45475A"
BorderThickness="1" CornerRadius="10">
                <TextBox Text="{x:Bind ViewModel.Name, Mode=TwoWay,
UpdateSourceTrigger=PropertyChanged}"
                    PlaceholderText="Persona name..."
BorderThickness="0" Background="Transparent"
                    Foreground="#CDD6F4" Padding="12,10"/>
            </Border>
        </StackPanel>
        <StackPanel Spacing="6" Width="200">
            <TextBlock Text="DESCRIPTION" FontSize="10"
FontWeight="SemiBold" CharacterSpacing="120" Foreground="#6C7086"/>
            <Border Background="#313244" BorderBrush="#45475A"
BorderThickness="1" CornerRadius="10">
                <TextBox Text="{x:Bind ViewModel.Description, Mode=TwoWay,
UpdateSourceTrigger=PropertyChanged}"
                    PlaceholderText="Short description..."
BorderThickness="0" Background="Transparent"
                    Foreground="#CDD6F4" Padding="12,10"/>
            </Border>
        </StackPanel>
    </StackPanel>
    <StackPanel Grid.Row="2" Spacing="6" Margin="0,0,0,14">
        <TextBlock Text="AVATAR COLOR" FontSize="10"
FontWeight="SemiBold" CharacterSpacing="120" Foreground="#6C7086"/>
        <StackPanel Orientation="Horizontal" Spacing="8"
x:Name="ColorPicker"/>

```

```

        </StackPanel>
        <StackPanel Grid.Row="3" Spacing="6">
            <TextBlock Text="SYSTEM PROMPT" FontSize="10"
FontWeight="SemiBold" CharacterSpacing="120" Foreground="#6C7086"/>
            <TextBlock FontSize="11" Foreground="#6C7086"
TextWrapping="Wrap">
                This is the instruction given to the AI at the start of every
conversation with this persona.
                Define their personality, expertise, tone, and any rules they
should follow.
            </TextBlock>
            <Border Background="#313244" BorderBrush="#45475A"
BorderThickness="1" CornerRadius="12" MinHeight="160">
                <TextBox Text="{x:Bind ViewModel.SystemPrompt, Mode=TwoWay,
UpdateSourceTrigger=PropertyChanged}"
                    PlaceholderText="You are a [role] who
[personality/expertise]..."
                    AcceptsReturn="True" TextWrapping="Wrap"
BorderThickness="0" Background="Transparent"
                    Foreground="#CDD6F4" Padding="14,12" FontSize="13"
MinHeight="160"/>
            </Border>
        </StackPanel>
        <StackPanel Grid.Row="4" Orientation="Horizontal" Spacing="12"
Margin="0,16,0,0">
            <Button Click="Save_Click" CornerRadius="10" Height="38"
Padding="18,0" BorderThickness="0">
                <Button.Background>
                    <LinearGradientBrush StartPoint="0,0" EndPoint="1,0">
                        <GradientStop Color="#7C6AF7" Offset="0"/>
                        <GradientStop Color="#F472B6" Offset="1"/>
                    </LinearGradientBrush>
                </Button.Background>
                <StackPanel Orientation="Horizontal" Spacing="8">
                    <FontIcon Glyph="⌂" FontSize="13" Foreground="White"/>
                    <TextBlock Text="Save Persona" Foreground="White"
FontWeight="SemiBold"/>
                </StackPanel>
            </Button>
            <Button Click="UseInChat_Click" CornerRadius="10" Height="38"
Padding="18,0"
                Background="#313244" BorderBrush="#45475A"
Foreground="#CDD6F4">
                <StackPanel Orientation="Horizontal" Spacing="8">
                    <FontIcon Glyph="💬" FontSize="13"
Foreground="#60A5FA"/>
                    <TextBlock Text="Chat with this Persona"/>
                </StackPanel>
            </Button>
            <Button Click="Delete_Click" CornerRadius="10" Height="38"
Padding="18,0"
                Background="#2D1A1A" BorderBrush="#4A2020"
Foreground="#F87171">
                <StackPanel Orientation="Horizontal" Spacing="8">

```

```

        <FontIcon Glyph="&#xE74D;" FontSize="13"
Foreground="#F87171"/>
        <TextBlock Text="Delete"/>
    </StackPanel>
</Button>
</StackPanel>
</Grid>
</Grid>
</Page>
'@ -Encoding UTF8

```

```

Set-Content "$root\Views\PersonasPage.xaml.cs" @"
using Microsoft.Extensions.DependencyInjection;
using Microsoft.UI.Xaml;
using Microsoft.UI.Xaml.Controls;
using Microsoft.UI.Xaml.Media;
using OllamaHub.Models;
using OllamaHub.ViewModels;
using Windows.UI;
namespace OllamaHub.Views;
public sealed partial class PersonasPage : Page
{
    public PersonaViewModel ViewModel { get; }
    public PersonasPage()
    {
        ViewModel = App.Services.GetRequiredService<PersonaViewModel>();
        InitializeComponent();
        Loaded += (_, _) => BuildColorPicker();
    }
    private void BuildColorPicker()
    {
        ColorPicker.Children.Clear();
        foreach (var color in ViewModel.AvatarColors)
        {
            var btn = new Button
            {
                Width=32, Height=32, CornerRadius=new CornerRadius(8),
                Background=new SolidColorBrush(Color.FromArgb(255,
                    Convert.ToByte(color[1..3],16),
                    Convert.ToByte(color[3..5],16), Convert.ToByte(color[5..7],16))),
                BorderThickness=new Thickness(2),
                Tag=color
            };
            btn.Click += (s, _) => { if(s is Button b)
ViewModel.AvatarColor = b.Tag?.ToString() ?? "#7C6AF7"; };
            ColorPicker.Children.Add(btn);
        }
    }
    private void PersonaList_SelectionChanged(object s,
SelectionChangedEventArgs e)
    {
        if (PersonaList.SelectedItem is Persona p)
ViewModel.SelectPersona(p);
    }
}

```



```

        <TextBlock Text="Connect prompts in a pipeline. Output of
each step feeds into the next."
        FontSize="11" Foreground="#6C7086"
TextWrapping="Wrap"/>
    </StackPanel>
</Border>
<StackPanel Grid.Row="1" Orientation="Horizontal" Spacing="8"
Margin="12,0,12,10">
    <Button Click="NewChain_Click" HorizontalAlignment="Stretch"
CornerRadius="10" Height="34" BorderThickness="0" Padding="12,0">
        <Button.Background>
            <LinearGradientBrush StartPoint="0,0" EndPoint="1,0">
                <GradientStop Color="#2DD4BF" Offset="0"/>
                <GradientStop Color="#60A5FA" Offset="1"/>
            </LinearGradientBrush>
        </Button.Background>
        <StackPanel Orientation="Horizontal" Spacing="6">
            <FontIcon Glyph="⌕" FontSize="12"
Foreground="White"/>
            <TextBlock Text="New Chain" Foreground="White"
FontSize="12" FontWeight="SemiBold"/>
        </StackPanel>
    </Button>
    <Button Click="DeleteChain_Click" Width="34" Height="34"
CornerRadius="10"
        Background="#2D1A1A" BorderBrush="#4A2020"
ToolTipService.ToolTip="Delete selected chain">
        <FontIcon Glyph="✖" FontSize="13"
Foreground="#F87171"/>
    </Button>
</StackPanel>
<ListView Grid.Row="2" x:Name="ChainList" ItemsSource="{x:Bind
ViewModel.Chains}"
        SelectionChanged="ChainList_SelectionChanged"
Background="Transparent" Margin="6,0,6,8">
    <ListView.ItemTemplate>
        <DataTemplate x:DataType="models:PromptChain">
            <Border Padding="10,8" CornerRadius="8">
                <StackPanel Spacing="3">
                    <TextBlock Text="{x:Bind Name}" FontSize="13"
FontWeight="SemiBold" Foreground="#CDD6F4"
TextTrimming="CharacterEllipsis"/>
                    <TextBlock Text="{x:Bind Nodes.Count,
Converter={StaticResource MessageCountConverter}}"
FontSize="11" Foreground="#6C7086"/>
                </StackPanel>
            </Border>
        </DataTemplate>
    </ListView.ItemTemplate>
    <ListView.ItemContainerStyle>
        <Style TargetType="ListViewItem">
            <Setter Property="HorizontalContentAlignment"
Value="Stretch"/>
            <Setter Property="CornerRadius" Value="8"/>
        </Style>
    </ListView.ItemContainerStyle>

```

```

        <Setter Property="Margin" Value="0,1"/>
        <Setter Property="Padding" Value="0"/>
    </Style>
</ListView.ItemContainerStyle>
</ListView>
</Grid>
</Border>

<!-- RIGHT: Chain editor -->
<Grid Grid.Column="1">
    <Grid.RowDefinitions>
        <RowDefinition Height="Auto"/>
        <RowDefinition Height="Auto"/>
        <RowDefinition Height="*" />
        <RowDefinition Height="Auto"/>
    </Grid.RowDefinitions>

    <!-- Header -->
    <Border Grid.Row="0" Background="#181825" BorderBrush="#313244"
BorderThickness="0,0,0,1" Padding="20,14">
        <Grid>
            <Grid.ColumnDefinitions>
                <ColumnDefinition Width="*" />
                <ColumnDefinition Width="Auto" />
            </Grid.ColumnDefinitions>
            <StackPanel Grid.Column="0" Spacing="4">
                <TextBlock Text="{x:Bind ViewModel.SelectedChain.Name,
Mode=OneWay, FallbackValue='Select a chain'}"
                    FontSize="18" FontWeight="SemiBold"
Foreground="#CDD6F4"/>
                <TextBlock Text="{x:Bind ViewModel.StatusText, Mode=OneWay}"
FontSize="12" Foreground="#6C7086"/>
            </StackPanel>
            <StackPanel Grid.Column="1" Orientation="Horizontal"
Spacing="10">
                <Button Click="AddNode_Click" CornerRadius="10" Height="36"
Padding="14,0"
                    Background="#313244" BorderBrush="#45475A"
Foreground="#CDD6F4" FontSize="12">
                    <StackPanel Orientation="Horizontal" Spacing="6">
                        <FontIcon Glyph="⌕" FontSize="12"
Foreground="#2DD4BF"/>
                        <TextBlock Text="Add Step"/>
                    </StackPanel>
                </Button>
                <Button Command="{x:Bind ViewModel.RunChainCommand}"
CornerRadius="10" Height="36" Padding="16,0"
                    IsEnabled="{x:Bind ViewModel.IsRunning, Mode=OneWay,
Converter={StaticResource InverseBoolConverter}}"
                    BorderThickness="0">
                    <Button.Background>
                        <LinearGradientBrush StartPoint="0,0" EndPoint="1,0">
                            <GradientStop Color="#2DD4BF" Offset="0"/>
                            <GradientStop Color="#60A5FA" Offset="1"/>

```

[illegible]


```

        <ColumnDefinition Width="Auto"/>
    </Grid.ColumnDefinitions>
    <Border Grid.Column="0" Background="#2DD4BF"
CornerRadius="6" Padding="8,4" Margin="0,0,10,0">
        <TextBlock Text="{x:Bind Order}" FontSize="12"
FontWeight="Bold" Foreground="#0F0F13"/>
    </Border>
    <TextBlock Grid.Column="1" Text="{x:Bind Name}"
FontSize="14" FontWeight="SemiBold"
        Foreground="#CDD6F4"
VerticalAlignment="Center"/>
    <StackPanel Grid.Column="2" Orientation="Horizontal"
Spacing="6">
        <ProgressRing IsActive="{x:Bind IsRunning}"
Width="16" Height="16" Foreground="#2DD4BF"/>
        <FontIcon Glyph="&#xE8FB;" FontSize="14"
Foreground="#4ADE80"
            Visibility="{x:Bind IsDone,
Converter={StaticResource BoolToVisibilityConverter}}"/>
    </StackPanel>
</Grid>
<!-- Prompt -->
<StackPanel Grid.Row="1" Spacing="4">
    <TextBlock Text="PROMPT (use {{PREV}} for previous
output, {{INPUT}} for initial input)"
        FontSize="10" FontWeight="SemiBold"
CharacterSpacing="80" Foreground="#6C7086"/>
    <Border Background="#1E1E2E" BorderBrush="#45475A"
BorderThickness="1" CornerRadius="8">
        <TextBox Text="{x:Bind Prompt}"
AcceptsReturn="True" TextWrapping="Wrap" MaxHeight="80"
            BorderThickness="0"
Background="Transparent" Foreground="#CDD6F4"
                Padding="10,8" FontSize="12"
FontFamily="Cascadia Code,Consolas,monospace"/>
    </Border>
</StackPanel>
<!-- Output label -->
<StackPanel Grid.Row="2" Spacing="4"
        Visibility="{x:Bind IsDone,
Converter={StaticResource BoolToVisibilityConverter}}">
    <TextBlock Text="OUTPUT" FontSize="10"
FontWeight="SemiBold" CharacterSpacing="80" Foreground="#6C7086"/>
    <Border Background="#1A2E1A" BorderBrush="#2A4A2A"
BorderThickness="1" CornerRadius="8" Padding="10,8" MaxHeight="120">
        <ScrollViewer VerticalScrollBarVisibility="Auto">
            <TextBlock Text="{x:Bind LastOutput}"
TextWrapping="Wrap" FontSize="12"
                Foreground="#A6E3A1"
IsTextSelectionEnabled="True"
                FontFamily="Cascadia
Code,Consolas,monospace"/>
        </ScrollViewer>
    </Border>

```

```

        </StackPanel>
    </Grid>
</Border>
</DataTemplate>
</ItemsControl.ItemTemplate>
</ItemsControl>
</ScrollView>

<!-- Empty state -->
<StackPanel Grid.Row="2" HorizontalAlignment="Center"
VerticalAlignment="Center" Spacing="12"
    Visibility="{x:Bind ViewModel.Nodes.Count, Mode=OneWay,
Converter={StaticResource ZeroToVisibilityConverter}}">
    <FontIcon Glyph="&#xE8EF;" FontSize="48" Foreground="#45475A"
HorizontalAlignment="Center"/>
    <TextBlock Text="Select a chain or create a new one"
FontSize="15" Foreground="#6C7086" HorizontalAlignment="Center"/>
    <TextBlock Text="Chains let you pipeline prompts together â€” the
output of each step becomes the input for the next"
        FontSize="13" Foreground="#45475A"
HorizontalAlignment="Center" TextWrapping="Wrap" MaxWidth="400"
TextAlignment="Center"/>
</StackPanel>
</Grid>
</Grid>
</Page>
'@ -Encoding UTF8

```

```

Set-Content "$root\Views\ChainPage.xaml.cs" '@
using Microsoft.Extensions.DependencyInjection;
using Microsoft.UI.Xaml;
using Microsoft.UI.Xaml.Controls;
using OllamaHub.Models;
using OllamaHub.ViewModels;
namespace OllamaHub.Views;
public sealed partial class ChainPage : Page
{
    public ChainViewModel ViewModel { get; }
    public ChainPage()
    {
        ViewModel = App.Services.GetRequiredService<ChainViewModel>();
        InitializeComponent();
    }
    private void ChainList_SelectionChanged(object s,
SelectionChangedEventArgs e)
    {
        if (ChainList.SelectedItem is PromptChain chain)
ViewModel.SelectChain(chain);
    }
    private void NewChain_Click(object s, RoutedEventArgs e) =>
ViewModel.NewChainCommand.Execute(null);
    private void DeleteChain_Click(object s, RoutedEventArgs e) =>
ViewModel.DeleteChainCommand.Execute(null);
}

```



```

        <TextBlock Text="2" FontSize="12" FontWeight="Bold"
Foreground="White" HorizontalAlignment="Center"
VerticalAlignment="Center"/>
    </Border>
    <TextBlock Text="Pull a model from the Models tab: try
llama3.2 or deepseek-coder-v2" FontSize="13" Foreground="#BAC2DE"
VerticalAlignment="Center"/>
    </StackPanel>
    <StackPanel Orientation="Horizontal" Spacing="12">
        <Border Background="#4ADE80" CornerRadius="6" Width="24"
Height="24">
            <TextBlock Text="3" FontSize="12" FontWeight="Bold"
Foreground="White" HorizontalAlignment="Center"
VerticalAlignment="Center"/>
        </Border>
        <TextBlock Text="Go to Chat, select your model, and start a
conversation" FontSize="13" Foreground="#BAC2DE"
VerticalAlignment="Center"/>
    </StackPanel>
</StackPanel>
</StackPanel>
</Border>

<!-- Features grid -->
<TextBlock Text="Features overview" FontSize="16"
FontWeight="SemiBold" Foreground="#CDD6F4"/>
<Grid ColumnSpacing="12" RowSpacing="12">
    <Grid.ColumnDefinitions>
        <ColumnDefinition Width="*" />
        <ColumnDefinition Width="*" />
    </Grid.ColumnDefinitions>
    <Grid.RowDefinitions>
        <RowDefinition Height="Auto" />
        <RowDefinition Height="Auto" />
        <RowDefinition Height="Auto" />
        <RowDefinition Height="Auto" />
        <RowDefinition Height="Auto" />
        <RowDefinition Height="Auto" />
    </Grid.RowDefinitions>

    <Border Grid.Row="0" Grid.Column="0" Background="#1A1A2E"
BorderBrush="#313244" BorderThickness="1" CornerRadius="12"
Padding="16,14">
        <StackPanel Spacing="8">
            <StackPanel Orientation="Horizontal" Spacing="10">
                <FontIcon Glyph="&#xE8BD;" FontSize="18"
Foreground="#7C6AF7"/>
                <TextBlock Text="Chat" FontSize="14" FontWeight="SemiBold"
Foreground="#CDD6F4"/>
            </StackPanel>
            <TextBlock Text="Full conversation interface with session
history, voice input, TTS output, and slash command templates. Type / to
see all prompt shortcuts." FontSize="12" Foreground="#6C7086"
TextWrapping="Wrap"/>

```

```

        </StackPanel>
    </Border>

    <Border Grid.Row="0" Grid.Column="1" Background="#1A1A2E"
    BorderBrush="#313244" BorderThickness="1" CornerRadius="12"
    Padding="16,14">
        <StackPanel Spacing="8">
            <StackPanel Orientation="Horizontal" Spacing="10">
                <FontIcon Glyph="⌘" FontSize="18"
                Foreground="#7C6AF7"/>
                <TextBlock Text="CoPilot" FontSize="14"
                FontWeight="SemiBold" Foreground="#CDD6F4"/>
            </StackPanel>
            <TextBlock Text="Dedicated code assistant. Paste code and
            choose Explain, Review, Fix, Write Tests, Generate Docs, or Refactor.
            Chat mode for general questions." FontSize="12" Foreground="#6C7086"
            TextWrapping="Wrap"/>
        </StackPanel>
    </Border>

    <Border Grid.Row="1" Grid.Column="0" Background="#1A1A2E"
    BorderBrush="#313244" BorderThickness="1" CornerRadius="12"
    Padding="16,14">
        <StackPanel Spacing="8">
            <StackPanel Orientation="Horizontal" Spacing="10">
                <FontIcon Glyph="⌘" FontSize="18"
                Foreground="#F472B6"/>
                <TextBlock Text="Personas" FontSize="14"
                FontWeight="SemiBold" Foreground="#CDD6F4"/>
            </StackPanel>
            <TextBlock Text="Create named AI characters with custom
            system prompts and personalities. Click 'Chat with this Persona' to start
            a conversation as that character." FontSize="12" Foreground="#6C7086"
            TextWrapping="Wrap"/>
        </StackPanel>
    </Border>

    <Border Grid.Row="1" Grid.Column="1" Background="#1A1A2E"
    BorderBrush="#313244" BorderThickness="1" CornerRadius="12"
    Padding="16,14">
        <StackPanel Spacing="8">
            <StackPanel Orientation="Horizontal" Spacing="10">
                <FontIcon Glyph="⌘" FontSize="18"
                Foreground="#2DD4BF"/>
                <TextBlock Text="Prompt Chains" FontSize="14"
                FontWeight="SemiBold" Foreground="#CDD6F4"/>
            </StackPanel>
            <TextBlock Text="Unique feature! Build a visual pipeline of
            prompts. Each step's output becomes the next step's input. Use {{PREV}}
            and {{INPUT}} in your prompts." FontSize="12" Foreground="#6C7086"
            TextWrapping="Wrap"/>
        </StackPanel>
    </Border>

```

```

        <Border Grid.Row="2" Grid.Column="0" Background="#1A1A2E"
BorderBrush="#313244" BorderThickness="1" CornerRadius="12"
Padding="16,14">
            <StackPanel Spacing="8">
                <StackPanel Orientation="Horizontal" Spacing="10">
                    <FontIcon Glyph="&#xE8C4;" FontSize="18"
Foreground="#60A5FA"/>
                    <TextBlock Text="Compare" FontSize="14"
FontWeight="SemiBold" Foreground="#CDD6F4"/>
                </StackPanel>
                <TextBlock Text="Run the same prompt on two models
simultaneously and compare responses, speed, and token count side by
side." FontSize="12" Foreground="#6C7086" TextWrapping="Wrap"/>
            </StackPanel>
        </Border>

        <Border Grid.Row="2" Grid.Column="1" Background="#1A1A2E"
BorderBrush="#313244" BorderThickness="1" CornerRadius="12"
Padding="16,14">
            <StackPanel Spacing="8">
                <StackPanel Orientation="Horizontal" Spacing="10">
                    <FontIcon Glyph="&#xE9D9;" FontSize="18"
Foreground="#4ADE80"/>
                    <TextBlock Text="Performance Stats" FontSize="14"
FontWeight="SemiBold" Foreground="#CDD6F4"/>
                </StackPanel>
                <TextBlock Text="Live dashboard showing tokens/second,
average latency, total tokens generated, and estimated cost savings vs
cloud APIs." FontSize="12" Foreground="#6C7086" TextWrapping="Wrap"/>
            </StackPanel>
        </Border>

        <Border Grid.Row="3" Grid.Column="0" Background="#1A1A2E"
BorderBrush="#313244" BorderThickness="1" CornerRadius="12"
Padding="16,14">
            <StackPanel Spacing="8">
                <StackPanel Orientation="Horizontal" Spacing="10">
                    <FontIcon Glyph="&#xE8F4;" FontSize="18"
Foreground="#FBBF24"/>
                    <TextBlock Text="Prompt Library" FontSize="14"
FontWeight="SemiBold" Foreground="#CDD6F4"/>
                </StackPanel>
                <TextBlock Text="Save and organize reusable prompt templates.
Access them instantly in chat by typing / followed by the template name."
FontSize="12" Foreground="#6C7086" TextWrapping="Wrap"/>
            </StackPanel>
        </Border>

        <Border Grid.Row="3" Grid.Column="1" Background="#1A1A2E"
BorderBrush="#313244" BorderThickness="1" CornerRadius="12"
Padding="16,14">
            <StackPanel Spacing="8">
                <StackPanel Orientation="Horizontal" Spacing="10">

```

```

        <FontIcon Glyph="#xE756;" FontSize="18"
Foreground="#A6E3A1"/>
        <TextBlock Text="Terminal" FontSize="14"
FontWeight="SemiBold" Foreground="#CDD6F4"/>
    </StackPanel>
    <TextBlock Text="Built-in command prompt with command history
(up/down arrows). Run Ollama commands, git, or any Windows command
without leaving the app." FontSize="12" Foreground="#6C7086"
TextWrapping="Wrap"/>
    </StackPanel>
</Border>
</Grid>

<!-- Keyboard shortcuts -->
<TextBlock Text="Keyboard shortcuts" FontSize="16"
FontWeight="SemiBold" Foreground="#CDD6F4"/>
<Border Background="#252535" BorderBrush="#313244"
BorderThickness="1" CornerRadius="14" Padding="20">
    <Grid ColumnSpacing="24" RowSpacing="10">
        <Grid.ColumnDefinitions>
            <ColumnDefinition Width="*"/>
            <ColumnDefinition Width="*"/>
        </Grid.ColumnDefinitions>
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
        </Grid.RowDefinitions>

        <StackPanel Grid.Row="0" Grid.Column="0"
Orientation="Horizontal" Spacing="12">
            <Border Background="#313244" CornerRadius="6"
Padding="8,4"><TextBlock Text="Enter" FontSize="12" Foreground="#CDD6F4"
FontFamily="Cascadia Code,Consolas,monospace"/></Border>
            <TextBlock Text="Send message" FontSize="13"
Foreground="#BAC2DE" VerticalAlignment="Center"/>
        </StackPanel>

        <StackPanel Grid.Row="0" Grid.Column="1"
Orientation="Horizontal" Spacing="12">
            <Border Background="#313244" CornerRadius="6"
Padding="8,4"><TextBlock Text="Ctrl+Enter" FontSize="12"
Foreground="#CDD6F4" FontFamily="Cascadia
Code,Consolas,monospace"/></Border>
            <TextBlock Text="New line in input" FontSize="13"
Foreground="#BAC2DE" VerticalAlignment="Center"/>
        </StackPanel>

        <StackPanel Grid.Row="1" Grid.Column="0"
Orientation="Horizontal" Spacing="12">
            <Border Background="#313244" CornerRadius="6"
Padding="8,4"><TextBlock Text="/" FontSize="12" Foreground="#CDD6F4"
FontFamily="Cascadia Code,Consolas,monospace"/></Border>

```

```

        <TextBlock Text="Open slash command menu" FontSize="13"
Foreground="#BAC2DE" VerticalAlignment="Center"/>
    </StackPanel>
    <StackPanel Grid.Row="1" Grid.Column="1"
Orientation="Horizontal" Spacing="12">
        <Border Background="#313244" CornerRadius="6"
Padding="8,4"><TextBlock Text="Esc" FontSize="12" Foreground="#CDD6F4"
FontFamily="Cascadia Code,Consolas,monospace"/></Border>
        <TextBlock Text="Close slash menu / cancel" FontSize="13"
Foreground="#BAC2DE" VerticalAlignment="Center"/>
    </StackPanel>
    <StackPanel Grid.Row="2" Grid.Column="0"
Orientation="Horizontal" Spacing="12">
        <Border Background="#313244" CornerRadius="6"
Padding="8,4"><TextBlock Text="â†` â†`" FontSize="12"
Foreground="#CDD6F4" FontFamily="Cascadia
Code,Consolas,monospace"/></Border>
        <TextBlock Text="Navigate slash menu / terminal history"
FontSize="13" Foreground="#BAC2DE" VerticalAlignment="Center"/>
    </StackPanel>
    <StackPanel Grid.Row="2" Grid.Column="1"
Orientation="Horizontal" Spacing="12">
        <Border Background="#313244" CornerRadius="6"
Padding="8,4"><TextBlock Text="Click Send again" FontSize="12"
Foreground="#CDD6F4" FontFamily="Cascadia
Code,Consolas,monospace"/></Border>
        <TextBlock Text="Stop generation mid-stream" FontSize="13"
Foreground="#BAC2DE" VerticalAlignment="Center"/>
    </StackPanel>
</Grid>
</Border>

<!-- Tips -->
<TextBlock Text="Pro tips" FontSize="16" FontWeight="SemiBold"
Foreground="#CDD6F4"/>
    <StackPanel Spacing="10">
        <Border Background="#1A2E1A" BorderBrush="#2A4A2A"
BorderThickness="1" CornerRadius="10" Padding="14,12">
            <StackPanel Orientation="Horizontal" Spacing="10">
                <FontIcon Glyph="&#xE82D;" FontSize="16"
Foreground="#4ADE80"/>
                <TextBlock TextWrapping="Wrap" FontSize="13"
Foreground="#A6E3A1" Margin="0,0,0,0">
                    <Run FontWeight="SemiBold">Best models for coding:</Run>
                    <Run> deepseek-coder-v2, codestral, qwen2.5-coder. Pull
them in the Models tab.</Run>
                </TextBlock>
            </StackPanel>
        </Border>
        <Border Background="#1A1A2E" BorderBrush="#313244"
BorderThickness="1" CornerRadius="10" Padding="14,12">
            <StackPanel Orientation="Horizontal" Spacing="10">
                <FontIcon Glyph="&#xE82D;" FontSize="16"
Foreground="#60A5FA"/>

```



```

        <TextBlock TextWrapping="Wrap" FontSize="13"
Foreground="#BAC2DE">
        <Run FontWeight="SemiBold">Prompt Chains tip:</Run>
        <Run> Use {{PREV}} in a step's prompt to insert the
previous step's output. Use {{INPUT}} to reference the initial input at
any step.</Run>
        </TextBlock>
    </StackPanel>
</Border>
    <Border Background="#1A1A2E" BorderBrush="#313244"
BorderThickness="1" CornerRadius="10" Padding="14,12">
        <StackPanel Orientation="Horizontal" Spacing="10">
            <FontIcon Glyph="&#xE82D;" FontSize="16"
Foreground="#F472B6"/>
            <TextBlock TextWrapping="Wrap" FontSize="13"
Foreground="#BAC2DE">
                <Run FontWeight="SemiBold">Personas tip:</Run>
                <Run> A well-written system prompt makes a huge difference.
Be specific about the persona's role, expertise level, communication
style, and any constraints.</Run>
            </TextBlock>
        </StackPanel>
    </Border>
    <Border Background="#1A1A2E" BorderBrush="#313244"
BorderThickness="1" CornerRadius="10" Padding="14,12">
        <StackPanel Orientation="Horizontal" Spacing="10">
            <FontIcon Glyph="&#xE82D;" FontSize="16"
Foreground="#FBBF24"/>
            <TextBlock TextWrapping="Wrap" FontSize="13"
Foreground="#BAC2DE">
                <Run FontWeight="SemiBold">Context window tip:</Run>
                <Run> The green/amber/red bar at the top of chat shows how
full the context window is. Start a new chat when it turns red to avoid
degraded responses.</Run>
            </TextBlock>
        </StackPanel>
    </Border>
</StackPanel>

    <!-- Version info -->
    <Border Background="#181825" BorderBrush="#313244"
BorderThickness="1" CornerRadius="10" Padding="16,12">
        <StackPanel Orientation="Horizontal" Spacing="12">
            <FontIcon Glyph="&#xE99A;" FontSize="20" Foreground="#7C6AF7"/>
            <StackPanel Spacing="2">
                <TextBlock Text="OllamaHub v3.0" FontSize="13"
FontWeight="SemiBold" Foreground="#CDD6F4"/>
                <TextBlock Text="Built on WinUI 3 Â· Powered by Ollama Â·
100% local Â· 100% free" FontSize="11" Foreground="#6C7086"/>
            </StackPanel>
        </StackPanel>
    </Border>
</StackPanel>
</ScrollViewer>

```

```

</Page>
'@ -Encoding UTF8

Set-Content "$root\Views\GuidePage.xaml.cs" @"
using Microsoft.UI.Xaml.Controls;
namespace OllamaHub.Views;
public sealed partial class GuidePage : Page
{
    public GuidePage() { InitializeComponent(); }
}
'@ -Encoding UTF8
Write-Ok "GuidePage"

# â"€â"€ Remaining pages (History, Models, Settings, Compare,
Performance, Terminal, Prompts) â"€â"€
Write-Step "Writing remaining views"

Set-Content "$root\Views\HistoryPage.xaml" @"
<?xml version="1.0" encoding="utf-8"?>
<Page x:Class="OllamaHub.Views.HistoryPage"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:models="using:OllamaHub.Models"
    Background="#1E1E2E">
    <Grid>
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="*/>
        </Grid.RowDefinitions>
        <Border Grid.Row="0" Background="#181825" BorderBrush="#313244"
BorderThickness="0,0,0,1" Padding="28,18">
            <Grid>
                <Grid.ColumnDefinitions><ColumnDefinition
Width="*/><ColumnDefinition Width="Auto"/></Grid.ColumnDefinitions>
                <StackPanel Grid.Column="0" Spacing="4">
                    <StackPanel Orientation="Horizontal" Spacing="12">
                        <Border Background="#313244" CornerRadius="8"
Padding="8"><FontIcon Glyph="&#xE81C;" FontSize="18"
Foreground="#60A5FA"/></Border>
                        <TextBlock Text="Chat History" FontSize="24"
FontWeight="Bold" Foreground="#CDD6F4" VerticalAlignment="Center"/>
                    </StackPanel>
                    <TextBlock Text="Search and manage your past conversations"
FontSize="13" Foreground="#6C7086" Margin="0,4,0,0"/>
                </StackPanel>
                <Border Grid.Column="1" Background="#313244"
BorderBrush="#45475A" BorderThickness="1" CornerRadius="10">
                    <TextBox Width="240" PlaceholderText="Search history..."
Text="{x:Bind ViewModel.SearchQuery, Mode=TwoWay,
UpdateSourceTrigger=PropertyChanged}"
BorderThickness="0" Background="Transparent"
Foreground="#CDD6F4" Padding="12,10"/>
                </Border>
            </Grid>

```

```

        </Border>
        <ScrollViewer Grid.Row="1" Padding="28,20">
            <ItemsControl ItemsSource="{x:Bind ViewModel.FilteredSessions,
Mode=OneWay}">
                <ItemsControl.ItemTemplate>
                    <DataTemplate x:DataType="models:ChatSession">
                        <Border Background="#252535" BorderBrush="#313244"
BorderThickness="1" CornerRadius="14" Padding="20,16" Margin="0,0,0,10">
                            <Grid>
                                <Grid.ColumnDefinitions><ColumnDefinition
Width="Auto"/><ColumnDefinition Width="*" /><ColumnDefinition
Width="Auto"/></Grid.ColumnDefinitions>
                                <Border Grid.Column="0" Width="4" CornerRadius="2"
Margin="0,0,16,0">
                                    <Border.Background>
                                        <LinearGradientBrush StartPoint="0,0" EndPoint="0,1">
                                            <GradientStop Color="#7C6AF7" Offset="0"/>
                                            <GradientStop Color="#60A5FA" Offset="1"/>
                                        </LinearGradientBrush>
                                    </Border.Background>
                                </Border>
                                <StackPanel Grid.Column="1" Spacing="6"
VerticalAlignment="Center">
                                    <TextBlock Text="{x:Bind Title}" FontSize="15"
FontWeight="SemiBold" Foreground="#CDD6F4"
TextTrimming="CharacterEllipsis"/>
                                    <StackPanel Orientation="Horizontal" Spacing="10">
                                        <Border Background="#313244" CornerRadius="5"
Padding="6,2"><TextBlock Text="{x:Bind ModelName}" FontSize="11"
Foreground="#7C6AF7"/></Border>
                                        <TextBlock Text="{x:Bind CreatedAt,
Converter={StaticResource DateToTextConverter}}" FontSize="11"
Foreground="#6C7086" VerticalAlignment="Center"/>
                                        <TextBlock Text="{x:Bind Messages.Count,
Converter={StaticResource MessageCountConverter}}" FontSize="11"
Foreground="#6C7086" VerticalAlignment="Center"/>
                                    </StackPanel>
                                </StackPanel>
                                <Button Grid.Column="2" Click="DeleteSession_Click"
Tag="{x:Bind}"
                                    Background="#2D1A1A" BorderBrush="#4A2020"
CornerRadius="8" Width="36" Height="36" Padding="0">
                                    <FontIcon Glyph="✕" FontSize="14"
Foreground="#F87171"/>
                                </Button>
                            </Grid>
                        </Border>
                    </DataTemplate>
                </ItemsControl.ItemTemplate>
            </ItemsControl>
        </ScrollViewer>
    </Grid>
</Page>
'@ -Encoding UTF8

```

```

Set-Content "$root\Views\HistoryPage.xaml.cs" @"
using Microsoft.Extensions.DependencyInjection;
using Microsoft.UI.Xaml;
using Microsoft.UI.Xaml.Controls;
using OllamaHub.Models;
using OllamaHub.ViewModels;
namespace OllamaHub.Views;
public sealed partial class HistoryPage : Page
{
    public HistoryViewModel ViewModel { get; }
    public HistoryPage() { ViewModel =
App.Services.GetRequiredService<HistoryViewModel>();
InitializeComponent(); }
    private void DeleteSession_Click(object s, RoutedEventArgs e) { if (s
is Button btn && btn.Tag is ChatSession session)
ViewModel.DeleteChatCommand.Execute(session); }
}
"@ -Encoding UTF8

```

```

Set-Content "$root\Views\ModelsPage.xaml" @"
<?xml version="1.0" encoding="utf-8"?>
<Page x:Class="OllamaHub.Views.ModelsPage"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:models="using:OllamaHub.Models"
    Background="#1E1E2E">
    <Grid>
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="*/"/>
        </Grid.RowDefinitions>
        <Border Grid.Row="0" Background="#181825" BorderBrush="#313244"
BorderThickness="0,0,0,1" Padding="28,18">
            <StackPanel Spacing="4">
                <StackPanel Orientation="Horizontal" Spacing="12">
                    <Border Background="#313244" CornerRadius="8"
Padding="8"><FontIcon Glyph="&#xE77B;" FontSize="18"
Foreground="#7C6AF7"/></Border>
                    <TextBlock Text="Models" FontSize="24" FontWeight="Bold"
Foreground="#CDD6F4" VerticalAlignment="Center"/>
                </StackPanel>
                <TextBlock Text="Manage your local Ollama models. Pull new ones
from ollama.com/library" FontSize="13" Foreground="#6C7086"
Margin="0,4,0,0"/>
            </StackPanel>
        </Border>
        <Border Grid.Row="1" Background="#181825" BorderBrush="#313244"
BorderThickness="0,0,0,1" Padding="28,14">
            <Grid ColumnSpacing="12">
                <Grid.ColumnDefinitions><ColumnDefinition
Width="*/"/><ColumnDefinition Width="Auto"/><ColumnDefinition
Width="Auto"/></Grid.ColumnDefinitions>

```

```

        <Border Grid.Column="0" Background="#313244"
BorderBrush="#45475A" BorderThickness="1" CornerRadius="10">
            <TextBox Text="{x:Bind ViewModel.PullModelName, Mode=TwoWay,
UpdateSourceTrigger=PropertyChanged}"
                PlaceholderText="Model name to pull (e.g. llama3.2,
deepseek-coder-v2, phi4)"
                BorderThickness="0" Background="Transparent"
Foreground="#CDD6F4" Padding="14,10"/>
            </Border>
            <Button Grid.Column="1" Command="{x:Bind
ViewModel.PullModelCommand}"
                IsEnabled="{x:Bind ViewModel.IsPulling, Mode=OneWay,
Converter={StaticResource InverseBoolConverter}}"
                CornerRadius="10" Height="42" Padding="16,0"
BorderThickness="0">
                <Button.Background>
                    <LinearGradientBrush StartPoint="0,0" EndPoint="1,0">
                        <GradientStop Color="#7C6AF7" Offset="0"/>
                        <GradientStop Color="#60A5FA" Offset="1"/>
                    </LinearGradientBrush>
                </Button.Background>
                <StackPanel Orientation="Horizontal" Spacing="8">
                    <FontIcon Glyph="⏮" FontSize="14" Foreground="White"/>
                    <TextBlock Text="Pull" Foreground="White"
FontWeight="SemiBold"/>
                </StackPanel>
            </Button>
            <Button Grid.Column="2" Command="{x:Bind
ViewModel.LoadModelsCommand}"
                CornerRadius="10" Height="42" Padding="14,0"
Background="#313244" BorderBrush="#45475A">
                <ToolTipService.ToolTip><ToolTip Content="Refresh model
list"/></ToolTipService.ToolTip>
                <FontIcon Glyph="🔄" FontSize="16" Foreground="#CDD6F4"/>
            </Button>
        </Grid>
    </Border>
    <ScrollViewer Grid.Row="2" Padding="28,20">
        <StackPanel Spacing="12">
            <ProgressBar Value="{x:Bind ViewModel.PullProgress, Mode=OneWay}"
Maximum="100" Foreground="#7C6AF7"
                Visibility="{x:Bind ViewModel.IsPulling,
Mode=OneWay, Converter={StaticResource BoolToVisibilityConverter}}"
Margin="0,0,0,4"/>
            <TextBlock Text="{x:Bind ViewModel.PullStatus, Mode=OneWay}"
FontSize="12" Foreground="#7C6AF7" Margin="0,0,0,8"
                Visibility="{x:Bind ViewModel.IsPulling, Mode=OneWay,
Converter={StaticResource BoolToVisibilityConverter}}"/>
            <ItemsControl ItemsSource="{x:Bind ViewModel.Models,
Mode=OneWay}">
                <ItemsControl.ItemTemplate>
                    <DataTemplate x:DataType="models:OllamaModelInfo">
                        <Border Background="#252535" BorderBrush="#313244"
BorderThickness="1" CornerRadius="12" Padding="18,14" Margin="0,0,0,8">

```

```

        <Grid>
            <Grid.ColumnDefinitions><ColumnDefinition
Width="*" /><ColumnDefinition Width="Auto" /></Grid.ColumnDefinitions>
            <StackPanel Grid.Column="0" Spacing="6">
                <TextBlock Text="{x:Bind Name}" FontSize="15"
FontWeight="SemiBold" Foreground="#CDD6F4" />
                <StackPanel Orientation="Horizontal" Spacing="10">
                    <Border Background="#313244" CornerRadius="5"
Padding="6,2"><TextBlock Text="{x:Bind DisplaySize}" FontSize="11"
Foreground="#60A5FA" /></Border>
                </StackPanel>
            </StackPanel>
            <Button Grid.Column="1" Click="DeleteModel_Click"
Tag="{x:Bind}"
                    Background="#2D1A1A" BorderBrush="#4A2020"
CornerRadius="8" Width="36" Height="36" Padding="0">
                <FontIcon Glyph="⌫" FontSize="14"
Foreground="#F87171" />
            </Button>
        </Grid>
    </Border>
</DataTemplate>
</ItemsControl.ItemTemplate>
</ItemsControl>
</StackPanel>
</ScrollViewer>
</Grid>
</Page>
'@ -Encoding UTF8

```

```

Set-Content "$root\Views\ModelsPage.xaml.cs" '@
using Microsoft.Extensions.DependencyInjection;
using Microsoft.UI.Xaml;
using Microsoft.UI.Xaml.Controls;
using OllamaHub.Models;
using OllamaHub.ViewModels;
namespace OllamaHub.Views;
public sealed partial class ModelsPage : Page
{
    public ModelsViewModel ViewModel { get; }
    public ModelsPage() { ViewModel =
App.Services.GetRequiredService<ModelsViewModel>();
InitializeComponent(); }
    private async void DeleteModel_Click(object s, RoutedEventArgs e)
    {
        if (s is Button btn && btn.Tag is OllamaModelInfo m)
        {
            var dlg = new ContentDialog { Title="Delete model",
Content=$"Delete {m.Name}? This cannot be undone.",
PrimaryButtonText="Delete", CloseButtonText="Cancel", XamlRoot=XamlRoot
};
            if (await dlg.ShowAsync() == ContentDialogResult.Primary)
ViewModel.DeleteModelCommand.Execute(m);
        }
    }
}

```

```

    }
}
'@ -Encoding UTF8

Set-Content "$root\Views\SettingsPage.xaml" '@'
<?xml version="1.0" encoding="utf-8"?>
<Page x:Class="OllamaHub.Views.SettingsPage"
      xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
      xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
      Background="#1E1E2E">
    <ScrollViewer VerticalScrollBarVisibility="Auto">
        <StackPanel Padding="32,28" Spacing="20" MaxWidth="640">
            <StackPanel Orientation="Horizontal" Spacing="12">
                <Border Background="#313244" CornerRadius="10"
                    Padding="10"><FontIcon Glyph="⚙️" FontSize="20"
                    Foreground="#7C6AF7"/></Border>
                <TextBlock Text="Settings" FontSize="24" FontWeight="Bold"
                    Foreground="#CDD6F4" VerticalAlignment="Center"/>
            </StackPanel>
            <Border Background="#252535" BorderBrush="#313244"
                BorderThickness="1" CornerRadius="14" Padding="20">
                <StackPanel Spacing="14">
                    <TextBlock Text="CONNECTION" FontSize="10"
                        FontWeight="SemiBold" CharacterSpacing="120" Foreground="#6C7086"/>
                    <StackPanel Spacing="6">
                        <TextBlock Text="Ollama base URL" FontSize="13"
                            Foreground="#9399B2"/>
                        <Border Background="#1E1E2E" BorderBrush="#45475A"
                            BorderThickness="1" CornerRadius="8">
                            <TextBox Text="{x:Bind ViewModel.OllamaUrl, Mode=TwoWay,
                                UpdateSourceTrigger=PropertyChanged}" BorderThickness="0"
                                Background="Transparent" Foreground="#CDD6F4" Padding="12,10"/>
                        </Border>
                    </StackPanel>
                    <StackPanel Spacing="6">
                        <TextBlock Text="Default model" FontSize="13"
                            Foreground="#9399B2"/>
                        <Border Background="#1E1E2E" BorderBrush="#45475A"
                            BorderThickness="1" CornerRadius="8">
                            <TextBox Text="{x:Bind ViewModel.DefaultModel, Mode=TwoWay,
                                UpdateSourceTrigger=PropertyChanged}" BorderThickness="0"
                                Background="Transparent" Foreground="#CDD6F4" Padding="12,10"/>
                        </Border>
                    </StackPanel>
                    <StackPanel Spacing="6">
                        <TextBlock Text="CoPilot preferred model (leave blank to
                            auto-detect code model)" FontSize="13" Foreground="#9399B2"/>
                        <Border Background="#1E1E2E" BorderBrush="#45475A"
                            BorderThickness="1" CornerRadius="8">
                            <TextBox Text="{x:Bind ViewModel.CopilotModel, Mode=TwoWay,
                                UpdateSourceTrigger=PropertyChanged}" PlaceholderText="e.g. deepseek-
                                    coder-v2" BorderThickness="0" Background="Transparent"
                                    Foreground="#CDD6F4" Padding="12,10"/>
                        </Border>
                    </StackPanel>
                </StackPanel>
            </Border>
        </StackPanel>
    </ScrollViewer>
</Page>

```

```

        </StackPanel>
    </StackPanel>
</Border>
<Border Background="#252535" BorderBrush="#313244"
BorderThickness="1" CornerRadius="14" Padding="20">
    <StackPanel Spacing="14">
        <TextBlock Text="GENERATION" FontSize="10"
FontWeight="SemiBold" CharacterSpacing="120" Foreground="#6C7086"/>
        <StackPanel Spacing="6">
            <TextBlock Text="{x:Bind ViewModel.Temperature, Mode=OneWay,
Converter={StaticResource FloatToLabelConverter},
ConverterParameter=Temperature}" FontSize="13" Foreground="#9399B2"/>
            <Slider Value="{x:Bind ViewModel.Temperature, Mode=TwoWay}"
Minimum="0" Maximum="2" StepFrequency="0.05"/>
        </StackPanel>
        <StackPanel Spacing="6">
            <TextBlock Text="Stream responses" FontSize="13"
Foreground="#9399B2"/>
            <ToggleSwitch IsOn="{x:Bind ViewModel.StreamResponses,
Mode=TwoWay}" />
        </StackPanel>
    </StackPanel>
</Border>
<Border Background="#252535" BorderBrush="#313244"
BorderThickness="1" CornerRadius="14" Padding="20">
    <StackPanel Spacing="14">
        <TextBlock Text="DEFAULT SYSTEM PROMPT" FontSize="10"
FontWeight="SemiBold" CharacterSpacing="120" Foreground="#6C7086"/>
        <Border Background="#1E1E2E" BorderBrush="#45475A"
BorderThickness="1" CornerRadius="8">
            <TextBox Text="{x:Bind ViewModel.SystemPrompt, Mode=TwoWay,
UpdateSourceTrigger=PropertyChanged}"
AcceptsReturn="True" TextWrapping="Wrap"
MinHeight="80" BorderThickness="0" Background="Transparent"
Foreground="#CDD6F4" Padding="12,10"/>
        </Border>
    </StackPanel>
</Border>
<StackPanel Orientation="Horizontal" Spacing="12" Margin="0,4,0,0">
    <Button Command="{x:Bind ViewModel.SaveCommand}"
CornerRadius="10" Height="40" Padding="20,0" BorderThickness="0">
        <Button.Background>
            <LinearGradientBrush StartPoint="0,0" EndPoint="1,0">
                <GradientStop Color="#7C6AF7" Offset="0"/>
                <GradientStop Color="#60A5FA" Offset="1"/>
            </LinearGradientBrush>
        </Button.Background>
        <TextBlock Text="Save settings" Foreground="White"
FontWeight="SemiBold"/>
    </Button>
    <TextBlock Text="{x:Bind ViewModel.SavedMessage, Mode=OneWay}"
FontSize="13" Foreground="#4ADE80" VerticalAlignment="Center"/>
</StackPanel>
</StackPanel>

```



```

    </ScrollView>
</Page>
'@ -Encoding UTF8
Set-Content "$root\Views\SettingsPage.xaml.cs" @"
using Microsoft.Extensions.DependencyInjection;
using Microsoft.UI.Xaml.Controls;
using OllamaHub.ViewModels;
namespace OllamaHub.Views;
public sealed partial class SettingsPage : Page
{
    public SettingsViewModel ViewModel { get; }
    public SettingsPage() { ViewModel =
App.Services.GetRequiredService<SettingsViewModel>();
InitializeComponent(); }
}
'@ -Encoding UTF8

Set-Content "$root\Views\PromptLibraryPage.xaml" @"
<?xml version="1.0" encoding="utf-8"?>
<Page x:Class="OllamaHub.Views.PromptLibraryPage"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:models="using:OllamaHub.Models"
    Background="#1E1E2E">
    <Grid>
        <Grid.ColumnDefinitions><ColumnDefinition
Width="280"/><ColumnDefinition Width="*" /></Grid.ColumnDefinitions>
        <Border Grid.Column="0" Background="#181825" BorderBrush="#313244"
BorderThickness="0,0,1,0">
            <Grid>
                <Grid.RowDefinitions><RowDefinition Height="Auto"/><RowDefinition
Height="Auto"/><RowDefinition Height="*" /></Grid.RowDefinitions>
                <StackPanel Grid.Row="0" Margin="14,14,14,10" Spacing="8">
                    <StackPanel Orientation="Horizontal" Spacing="10">
                        <Border Background="#313244" CornerRadius="8"
Padding="7"><FontIcon Glyph="&#xE8F4;" FontSize="16"
Foreground="#FBBF24"/></Border>
                        <TextBlock Text="Prompt Library" FontSize="15"
FontWeight="SemiBold" Foreground="#CDD6F4" VerticalAlignment="Center"/>
                    </StackPanel>
                    <Border Background="#313244" BorderBrush="#45475A"
BorderThickness="1" CornerRadius="10">
                        <TextBox PlaceholderText="Search..." Text="{x:Bind
ViewModel.SearchText, Mode=TwoWay, UpdateSourceTrigger=PropertyChanged}"
BorderThickness="0" Background="Transparent" Foreground="#CDD6F4"
Padding="12,9"/>
                    </Border>
                </StackPanel>
                <Button Grid.Row="1" Click="AddNew_Click"
HorizontalAlignment="Stretch" Margin="14,0,14,10" CornerRadius="10"
Height="36" BorderThickness="0">
                    <Button.Background><LinearGradientBrush StartPoint="0,0"
EndPoint="1,0"><GradientStop Color="#FBBF24" Offset="0"/><GradientStop
Color="#F97316" Offset="1"/></LinearGradientBrush></Button.Background>

```

```

        <StackPanel Orientation="Horizontal" Spacing="8"><FontIcon
Glyph="⌂" FontSize="13" Foreground="White"/><TextBlock Text="New
Prompt" Foreground="White" FontWeight="SemiBold"
FontSize="13"/></StackPanel>
    </Button>
    <ListView Grid.Row="2" x:Name="TemplateList" ItemsSource="{x:Bind
ViewModel.Filtered, Mode=OneWay}"
SelectionChanged="TemplateList_SelectionChanged" Background="Transparent"
Margin="6,0,6,8">
        <ListView.ItemTemplate>
            <DataTemplate x:DataType="models:PromptTemplate">
                <Border Padding="10,8" CornerRadius="8">
                    <Grid ColumnSpacing="10">
                        <Grid.ColumnDefinitions><ColumnDefinition
Width="Auto"/><ColumnDefinition Width="*" /></Grid.ColumnDefinitions>
                        <Border Grid.Column="0" Background="#313244"
CornerRadius="8" Width="32" Height="32">
                            <FontIcon Glyph="{x:Bind Icon}" FontSize="15"
Foreground="#FBBF24" HorizontalAlignment="Center"
VerticalAlignment="Center"/>
                        </Border>
                        <StackPanel Grid.Column="1" Spacing="2"
VerticalAlignment="Center">
                            <TextBlock Text="{x:Bind Name}" FontSize="13"
FontWeight="SemiBold" Foreground="#CDD6F4"
TextTrimming="CharacterEllipsis"/>
                            <TextBlock Text="{x:Bind Category}" FontSize="11"
Foreground="#6C7086"/>
                        </StackPanel>
                    </Grid>
                </Border>
            </DataTemplate>
        </ListView.ItemTemplate>
        <ListView.ItemContainerStyle>
            <Style TargetType="ListViewItem"><Setter
Property="HorizontalContentAlignment" Value="Stretch"/><Setter
Property="CornerRadius" Value="8"/><Setter Property="Margin"
Value="0,1"/><Setter Property="Padding" Value="0"/></Style>
        </ListView.ItemContainerStyle>
    </ListView>
</Grid>
</Border>
<Grid Grid.Column="1" Padding="28">
    <Grid.RowDefinitions><RowDefinition Height="Auto"/><RowDefinition
Height="Auto"/><RowDefinition Height="Auto"/><RowDefinition
Height="*" /></Grid.RowDefinitions>
    <TextBlock Grid.Row="0" Text="Edit Prompt" FontSize="22"
FontWeight="Bold" Foreground="#CDD6F4" Margin="0,0,0,20"/>
    <StackPanel Grid.Row="1" Orientation="Horizontal" Spacing="16"
Margin="0,0,0,14">
        <StackPanel Spacing="6" Width="220">
            <TextBlock Text="NAME" FontSize="10" FontWeight="SemiBold"
CharacterSpacing="120" Foreground="#6C7086"/>

```

```

        <Border Background="#313244" BorderBrush="#45475A"
BorderThickness="1" CornerRadius="10">
            <TextBox Text="{x:Bind ViewModel.NewName, Mode=TwoWay,
UpdateSourceTrigger=PropertyChanged}" PlaceholderText="Prompt name..."
BorderThickness="0" Background="Transparent" Foreground="#CDD6F4"
Padding="12,10"/>
        </Border>
    </StackPanel>
    <StackPanel Spacing="6" Width="160">
        <TextBlock Text="CATEGORY" FontSize="10" FontWeight="SemiBold"
CharacterSpacing="120" Foreground="#6C7086"/>
        <ComboBox x:Name="CategoryBox" ItemsSource="{x:Bind
ViewModel.Categories}" SelectedItem="{x:Bind ViewModel.NewCategory,
Mode=TwoWay}" HorizontalAlignment="Stretch" Background="#313244"
BorderBrush="#45475A"/>
    </StackPanel>
</StackPanel>
    <TextBlock Grid.Row="2" Text="CONTENT" FontSize="10"
FontWeight="SemiBold" CharacterSpacing="120" Foreground="#6C7086"
Margin="0,0,0,6"/>
    <Border Grid.Row="3" Background="#313244" BorderBrush="#45475A"
BorderThickness="1" CornerRadius="12">
        <TextBox Text="{x:Bind ViewModel.NewContent, Mode=TwoWay,
UpdateSourceTrigger=PropertyChanged}" PlaceholderText="Write your prompt
template here..." AcceptsReturn="True" TextWrapping="Wrap"
BorderThickness="0" Background="Transparent" Foreground="#CDD6F4"
Padding="16,14" FontFamily="Cascadia Code, Consolas, monospace"
FontSize="13" VerticalAlignment="Stretch"/>
    </Border>
    <StackPanel Grid.Row="4" Orientation="Horizontal" Spacing="12"
Margin="0,16,0,0">
        <Button Click="Save_Click" CornerRadius="10" Height="38"
Padding="16,0" BorderThickness="0">
            <Button.Background><LinearGradientBrush StartPoint="0,0"
EndPoint="1,0"><GradientStop Color="#FBBF24" Offset="0"/><GradientStop
Color="#F97316" Offset="1"/></LinearGradientBrush></Button.Background>
            <StackPanel Orientation="Horizontal" Spacing="8"><FontIcon
Glyph="&#xE74E;" FontSize="13" Foreground="White"/><TextBlock Text="Save"
Foreground="White" FontWeight="SemiBold"/></StackPanel>
        </Button>
        <Button Click="UsePrompt_Click" CornerRadius="10" Height="38"
Padding="16,0" Background="#313244" BorderBrush="#45475A"
Foreground="#CDD6F4">
            <StackPanel Orientation="Horizontal" Spacing="8"><FontIcon
Glyph="&#xE8BD;" FontSize="13" Foreground="#60A5FA"/><TextBlock Text="Use
in Chat"/></StackPanel>
        </Button>
        <Button Click="Delete_Click" CornerRadius="10" Height="38"
Padding="16,0" Background="#2D1A1A" BorderBrush="#4A2020"
Foreground="#F87171">
            <StackPanel Orientation="Horizontal" Spacing="8"><FontIcon
Glyph="&#xE74D;" FontSize="13" Foreground="#F87171"/><TextBlock
Text="Delete"/></StackPanel>
        </Button>

```

```

        </StackPanel>
    </Grid>
</Grid>
</Page>
'@ -Encoding UTF8
Set-Content "$root\Views\PromptLibraryPage.xaml.cs" '@'
using Microsoft.Extensions.DependencyInjection;
using Microsoft.UI.Xaml;
using Microsoft.UI.Xaml.Controls;
using OllamaHub.Models;
using OllamaHub.ViewModels;
namespace OllamaHub.Views;
public sealed partial class PromptLibraryPage : Page
{
    public PromptLibraryViewModel ViewModel { get; }
    public PromptLibraryPage() { ViewModel =
App.Services.GetRequiredService<PromptLibraryViewModel>();
InitializeComponent(); }
    private void TemplateList_SelectionChanged(object s,
SelectionChangedEventArgs e) { if (TemplateList.SelectedItem is
PromptTemplate t) { ViewModel.Selected=t; ViewModel.NewName=t.Name;
ViewModel.NewCategory=t.Category; ViewModel.NewContent=t.Content; } }
    private void AddNew_Click(object s, RoutedEventArgs e) {
ViewModel.Selected=null;
ViewModel.NewName=ViewModel.NewContent=string.Empty;
ViewModel.NewCategory="General"; TemplateList.SelectedItem=null; }
    private void Save_Click(object s, RoutedEventArgs e) {
if(ViewModel.Selected!=null){ViewModel.Selected.Name=ViewModel.NewName;Vi
ewModel.Selected.Category=ViewModel.NewCategory;ViewModel.Selected.Conten
t=ViewModel.NewContent;ViewModel.SaveSelectedCommand.Execute(null);}else
ViewModel.AddTemplateCommand.Execute(null); }
    private void Delete_Click(object s, RoutedEventArgs e) {
if(ViewModel.Selected!=null)
ViewModel.DeleteTemplateCommand.Execute(ViewModel.Selected); }
    private void UsePrompt_Click(object s, RoutedEventArgs e) {
ChatPage.PendingPrompt=ViewModel.NewContent; if(Frame.Parent is Frame pf)
pf.Navigate(typeof(ChatPage)); }
}
'@ -Encoding UTF8

Set-Content "$root\Views\ModelComparePage.xaml" '@'
<?xml version="1.0" encoding="utf-8"?>
<Page x:Class="OllamaHub.Views.ModelComparePage"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    Background="#1E1E2E">
    <Grid Padding="28" RowSpacing="16">
<Grid.RowDefinitions>
<RowDefinition Height="Auto"/>
<RowDefinition Height="Auto"/>
<RowDefinition Height="Auto"/>
<RowDefinition Height="*/>
</Grid.RowDefinitions>
<StackPanel Grid.Row="0" Orientation="Horizontal" Spacing="14">

```

```

<Border Background="#313244" CornerRadius="10" Padding="10"><FontIcon
Glyph="⚡" FontSize="22" Foreground="#60A5FA"/></Border>
<StackPanel VerticalAlignment="Center" Spacing="2">
<TextBlock Text="Model Comparison" FontSize="24" FontWeight="Bold"
Foreground="#CDD6F4"/>
<TextBlock Text="Same prompt, two models, side by side" FontSize="12"
Foreground="#6C7086"/>
</StackPanel>
</StackPanel>
<Grid Grid.Row="1" ColumnSpacing="16">
<Grid.ColumnDefinitions>
<ColumnDefinition Width="*" />
<ColumnDefinition Width="*" />
</Grid.ColumnDefinitions>
<Border Grid.Column="0" Background="#181825" BorderBrush="#313244"
BorderThickness="1" CornerRadius="12" Padding="14,12">
<StackPanel Spacing="6">
<TextBlock Text="MODEL A" FontSize="10" FontWeight="SemiBold"
CharacterSpacing="120" Foreground="#6C7086"/>
<ComboBox x:Name="ModelABox" ItemsSource="{x:Bind
ViewModel.AvailableModels}"
DisplayMemberPath="Name" SelectedIndex="0" Background="#313244"
BorderBrush="#45475A"
PlaceholderText="Select model A...">
<ToolTipService.ToolTip><ToolTip Content="Choose the first model to
compare"/></ToolTipService.ToolTip>
</ComboBox>
</StackPanel>
</Border>
<Border Grid.Column="1" Background="#181825" BorderBrush="#313244"
BorderThickness="1" CornerRadius="12" Padding="14,12">
<StackPanel Spacing="6">
<TextBlock Text="MODEL B" FontSize="10" FontWeight="SemiBold"
CharacterSpacing="120" Foreground="#6C7086"/>
<ComboBox x:Name="ModelBBox" ItemsSource="{x:Bind
ViewModel.AvailableModels}"
DisplayMemberPath="Name" SelectedIndex="1" Background="#313244"
BorderBrush="#45475A"
PlaceholderText="Select model B...">
<ToolTipService.ToolTip><ToolTip Content="Choose the second model to
compare"/></ToolTipService.ToolTip>
</ComboBox>
</StackPanel>
</Border>
</Grid>
<Border Grid.Row="2" Background="#181825" BorderBrush="#313244"
BorderThickness="1" CornerRadius="12" Padding="16,14">
<StackPanel Spacing="6">
<TextBlock Text="PROMPT" FontSize="10" FontWeight="SemiBold"
CharacterSpacing="120" Foreground="#6C7086"/>
<Border Background="#313244" BorderBrush="#45475A" BorderThickness="1"
CornerRadius="8">
<TextBox x:Name="PromptBox" Text="{x:Bind ViewModel.Prompt, Mode=TwoWay,
UpdateSourceTrigger=PropertyChanged}"

```

```

PlaceholderText="Enter your prompt here (both models will run the same
prompt)..."
AcceptsReturn="True" TextWrapping="Wrap" BorderThickness="0"
Background="Transparent"
Foreground="#CDD6F4" Padding="12,10" FontSize="13"/>
</Border>
</StackPanel>
</Border>
<Grid Grid.Row="3">
<Grid.ColumnDefinitions>
<ColumnDefinition Width="*" />
<ColumnDefinition Width="*" />
</Grid.ColumnDefinitions>
<!-- Left: Model A output -->
<Border Grid.Column="0" Margin="0,0,8,0" Background="#181825"
BorderBrush="#313244" BorderThickness="1" CornerRadius="12">
<Grid>
<Grid.RowDefinitions>
<RowDefinition Height="Auto" />
<RowDefinition Height="*" />
<RowDefinition Height="Auto" />
</Grid.RowDefinitions>
<Border Grid.Row="0" Padding="12,8" BorderBrush="#313244"
BorderThickness="0,0,0,1">
<Grid>
<Grid.ColumnDefinitions>
<ColumnDefinition Width="*" />
<ColumnDefinition Width="Auto" />
</Grid.ColumnDefinitions>
<TextBlock Text="Model A" FontSize="13" FontWeight="SemiBold"
Foreground="#CDD6F4" VerticalAlignment="Center" />
<TextBlock Grid.Column="1" Text="{x:Bind ViewModel.MsA,
Converter={StaticResource MsToSecConverter}, Mode=OneWay}"
FontSize="12" Foreground="#6C7086" VerticalAlignment="Center" />
</Grid>
</Border>
<ScrollViewer Grid.Row="1" VerticalScrollBarVisibility="Auto"
Padding="12">
<TextBlock Text="{x:Bind ViewModel.ResponseA, Mode=OneWay}"
TextWrapping="Wrap" FontSize="13" Foreground="#CDD6F4"
IsTextSelectionEnabled="True" LineHeight="20"
FontFamily="Cascadia Code, Consolas, monospace" />
</ScrollViewer>
<Border Grid.Row="2" Padding="12,6" BorderBrush="#313244"
BorderThickness="0,1,0,0" Background="#13131E">
<TextBlock Text="{x:Bind ViewModel.TokensA, Mode=OneWay,
StringFormat='{{0}} tokens'"
FontSize="11" Foreground="#6C7086" HorizontalAlignment="Right" />
</Border>
</Grid>
</Border>
<!-- Right: Model B output -->
<Border Grid.Column="1" Margin="8,0,0,0" Background="#181825"
BorderBrush="#313244" BorderThickness="1" CornerRadius="12">

```

```

<Grid>
<Grid.RowDefinitions>
<RowDefinition Height="Auto"/>
<RowDefinition Height="*/>
<RowDefinition Height="Auto"/>
</Grid.RowDefinitions>
<Border Grid.Row="0" Padding="12,8" BorderBrush="#313244"
BorderThickness="0,0,0,1">
<Grid>
<Grid.ColumnDefinitions>
<ColumnDefinition Width="*/>
<ColumnDefinition Width="Auto"/>
</Grid.ColumnDefinitions>
<TextBlock Text="Model B" FontSize="13" FontWeight="SemiBold"
Foreground="#CDD6F4" VerticalAlignment="Center"/>
<TextBlock Grid.Column="1" Text="{x:Bind ViewModel.MsB,
Converter={StaticResource MsToSecConverter}, Mode=OneWay}"
FontSize="12" Foreground="#6C7086" VerticalAlignment="Center"/>
</Grid>
</Border>
<ScrollViewer Grid.Row="1" VerticalScrollBarVisibility="Auto"
Padding="12">
<TextBlock Text="{x:Bind ViewModel.ResponseB, Mode=OneWay}"
TextWrapping="Wrap" FontSize="13" Foreground="#CDD6F4"
IsTextSelectionEnabled="True" LineHeight="20"
FontFamily="Cascadia Code,Consolas,monospace"/>
</ScrollViewer>
<Border Grid.Row="2" Padding="12,6" BorderBrush="#313244"
BorderThickness="0,1,0,0" Background="#13131E">
<TextBlock Text="{x:Bind ViewModel.TokensB, Mode=OneWay,
StringFormat='{}{0} tokens'}"
FontSize="11" Foreground="#6C7086" HorizontalAlignment="Right"/>
</Border>
</Grid>
</Border>
</Grid>
<Border Grid.Row="3" Background="#181825" BorderBrush="#313244"
BorderThickness="0,1,0,0" Padding="0,12,0,12"
HorizontalAlignment="Stretch">
<Grid>
<Grid.ColumnDefinitions>
<ColumnDefinition Width="*/>
<ColumnDefinition Width="Auto"/>
<ColumnDefinition Width="Auto"/>
</Grid.ColumnDefinitions>
<TextBlock Grid.Column="0" Text="{x:Bind ViewModel.StatusText,
Mode=OneWay}" FontSize="12" Foreground="#6C7086" Margin="0,0,12,0"/>
<Button Grid.Column="1" Click="ClearAll_Click" Content="Clear"
HorizontalAlignment="Right"
CornerRadius="8" Height="36" Background="#313244" BorderBrush="#45475A"
Margin="0,0,12,0"/>
<Button Grid.Column="2" Command="{x:Bind ViewModel.RunCompareCommand}"
CornerRadius="8" Height="36" Padding="16,0"

```



```

</Grid.RowDefinitions>
<Border Grid.Row="0" Background="#181825" BorderBrush="#313244"
BorderThickness="0,0,0,1" Padding="28,18">
<Grid>
<Grid.ColumnDefinitions>
<ColumnDefinition Width="*" />
<ColumnDefinition Width="Auto" />
</Grid.ColumnDefinitions>
<StackPanel Grid.Column="0" Spacing="4">
<StackPanel Orientation="Horizontal" Spacing="12">
<Border Background="#313244" CornerRadius="8" Padding="8"><FontIcon
Glyph="&#xE9D9;" FontSize="18" Foreground="#4ADE80"/></Border>
<TextBlock Text="Performance Dashboard" FontSize="24" FontWeight="Bold"
Foreground="#CDD6F4" VerticalAlignment="Center"/>
</StackPanel>
<TextBlock Text="Live metrics for your Ollama sessions" FontSize="13"
Foreground="#6C7086" Margin="0,4,0,0"/>
</StackPanel>
<Button Grid.Column="1" Command="{x:Bind ViewModel.CheckStatusCommand}"
CornerRadius="10" Height="42" Padding="14,0" Background="#313244"
BorderBrush="#45475A">
<ToolTipService.ToolTip><ToolTip Content="Check Ollama
status"/></ToolTipService.ToolTip>
<FontIcon Glyph="&#xE72C;" FontSize="16" Foreground="#CDD6F4"/>
</Button>
</Grid>
</Border>
<ScrollViewer Grid.Row="1" Padding="28,20">
<Grid RowSpacing="20">
<Grid.RowDefinitions>
<RowDefinition Height="Auto" />
<RowDefinition Height="Auto" />
<RowDefinition Height="Auto" />
<RowDefinition Height="Auto" />
<RowDefinition Height="*" />
</Grid.RowDefinitions>
<!-- Summary cards -->
<Grid ColumnSpacing="16">
<Grid.ColumnDefinitions>
<ColumnDefinition Width="*" />
<ColumnDefinition Width="*" />
<ColumnDefinition Width="*" />
</Grid.ColumnDefinitions>
<Border Background="#252535" BorderBrush="#313244" BorderThickness="1"
CornerRadius="14" Padding="20,18">
<StackPanel Spacing="8">
<TextBlock Text="TOKENS/SEC" FontSize="10" FontWeight="SemiBold"
CharacterSpacing="120" Foreground="#6C7086"/>
<TextBlock Text="{x:Bind ViewModel.CurrentTps, Mode=OneWay,
StringFormat='{0:N1}}'" FontSize="24" FontWeight="Bold"
Foreground="#4ADE80"/>
<TextBlock Text="Current tokens per second" FontSize="11"
Foreground="#6C7086"/>
</StackPanel>

```

```

</Border>
<Border Background="#252535" BorderBrush="#313244" BorderThickness="1"
CornerRadius="14" Padding="20,18">
<StackPanel Spacing="8">
<TextBlock Text="LATENCY" FontSize="10" FontWeight="SemiBold"
CharacterSpacing="120" Foreground="#6C7086"/>
<TextBlock Text="{x:Bind ViewModel.AvgLatencyMs, Mode=OneWay,
StringFormat='{0:N0} ms'}" FontSize="24" FontWeight="Bold"
Foreground="#60A5FA"/>
<TextBlock Text="Average response latency" FontSize="11"
Foreground="#6C7086"/>
</StackPanel>
</Border>
<Border Background="#252535" BorderBrush="#313244" BorderThickness="1"
CornerRadius="14" Padding="20,18">
<StackPanel Spacing="8">
<TextBlock Text="SAVINGS" FontSize="10" FontWeight="SemiBold"
CharacterSpacing="120" Foreground="#6C7086"/>
<TextBlock Text="{x:Bind ViewModel.EstimatedSavings, Mode=OneWay,
StringFormat='{0:N2}'}" FontSize="24" FontWeight="Bold"
Foreground="#F472B6"/>
<TextBlock Text="Estimated cost savings" FontSize="11"
Foreground="#6C7086"/>
</StackPanel>
</Border>
</Grid>
<!-- Stats -->
<Grid ColumnSpacing="16">
<Grid.ColumnDefinitions>
<ColumnDefinition Width="*" />
<ColumnDefinition Width="*" />
</Grid.ColumnDefinitions>
<Border Background="#252535" BorderBrush="#313244" BorderThickness="1"
CornerRadius="14" Padding="20,18">
<StackPanel Spacing="8">
<TextBlock Text="TOKENS" FontSize="10" FontWeight="SemiBold"
CharacterSpacing="120" Foreground="#6C7086"/>
<TextBlock Text="{x:Bind ViewModel.TotalTokens, Mode=OneWay}"
FontSize="24" FontWeight="Bold" Foreground="#FBBF24"/>
<TextBlock Text="Total tokens generated" FontSize="11"
Foreground="#6C7086"/>
</StackPanel>
</Border>
<Border Background="#252535" BorderBrush="#313244" BorderThickness="1"
CornerRadius="14" Padding="20,18">
<StackPanel Spacing="8">
<TextBlock Text="UPTIME" FontSize="10" FontWeight="SemiBold"
CharacterSpacing="120" Foreground="#6C7086"/>
<TextBlock Text="{x:Bind ViewModel.UptimeMinutes, Mode=OneWay,
StringFormat='{0:N1} min'}" FontSize="24" FontWeight="Bold"
Foreground="#2DD4BF"/>
<TextBlock Text="Total uptime" FontSize="11" Foreground="#6C7086"/>
</StackPanel>
</Border>

```

```

</Grid>
<!-- Samples -->
<Border Background="#252535" BorderBrush="#313244" BorderThickness="1"
CornerRadius="14" Padding="20">
<Grid>
<Grid.RowDefinitions>
<RowDefinition Height="Auto"/>
<RowDefinition Height="Auto"/>
<RowDefinition Height="*/>
</Grid.RowDefinitions>
<TextBlock Text="RECENT SAMPLES" FontSize="12" FontWeight="SemiBold"
Foreground="#CDD6F4" Margin="0,0,0,10"/>
<Border Height="1" Background="#45475A" Margin="0,0,0,10"/>
<ScrollView Grid.Row="2" VerticalScrollBarVisibility="Auto"
Padding="0,5,0,0">
<ItemsControl ItemsSource="{x:Bind ViewModel.Samples, Mode=OneWay}">
<ItemsControl.ItemTemplate>
<DataTemplate x:DataType="models:PerformanceSample">
<Border Background="#1E1E2E" BorderBrush="#313244" BorderThickness="1"
CornerRadius="8" Padding="12,10" Margin="0,0,0,8">
<Grid>
<Grid.ColumnDefinitions>
<ColumnDefinition Width="*/>
<ColumnDefinition Width="Auto"/>
</Grid.ColumnDefinitions>
<StackPanel Grid.Column="0" Spacing="4">
<TextBlock Text="{x:Bind ModelName}" FontSize="13" FontWeight="SemiBold"
Foreground="#CDD6F4"/>
<TextBlock Text="{x:Bind Timestamp, Converter={StaticResource
DateToTextConverter}, Mode=OneWay}" FontSize="11" Foreground="#6C7086"/>
</StackPanel>
<StackPanel Grid.Column="1" Spacing="4" HorizontalAlignment="Right">
<TextBlock Text="{x:Bind TokensPerSec, Mode=OneWay,
StringFormat='{}{0:N1} t/s'}" FontSize="12" Foreground="#4ADE80"/>
<TextBlock Text="{x:Bind LatencyMs, Mode=OneWay, StringFormat='{}{0:N0}
ms'}" FontSize="12" Foreground="#60A5FA"/>
</StackPanel>
</Grid>
</Border>
</DataTemplate>
</ItemsControl.ItemTemplate>
</ItemsControl>
</ScrollView>
</Grid>
</Border>
</Grid>
</ScrollView>
</Grid>
</Page>
'@ -Encoding UTF8

```

```

Set-Content "$root\Views\PerformancePage.xaml.cs" '@'
using Microsoft.Extensions.DependencyInjection;
using Microsoft.UI.Xaml.Controls;

```



```

<ScrollViewer Grid.Row="1" Padding="28,20" Background="#1E1E2E"
BorderBrush="#313244" BorderThickness="0,0,0,1">
<TextBlock x:Name="OutputBox" TextWrapping="Wrap" FontSize="13"
Foreground="#CDD6F4"
FontFamily="Cascadia Code,Consolas,monospace" LineHeight="20"/>
</ScrollViewer>
<Border Grid.Row="2" Background="#181825" BorderBrush="#313244"
BorderThickness="0,1,0,0" Padding="28,12">
<Grid ColumnSpacing="10">
<Grid.ColumnDefinitions>
<ColumnDefinition Width="Auto"/>
<ColumnDefinition Width="*" />
<ColumnDefinition Width="Auto"/>
</Grid.ColumnDefinitions>
<TextBlock Grid.Column="0" Text="$" FontSize="14" Foreground="#6C7086"
VerticalAlignment="Center"/>
<Border Grid.Column="1" Background="#313244" BorderBrush="#45475A"
BorderThickness="1" CornerRadius="10">
<TextBox x:Name="CommandBox"
TextChanged="CommandBox_TextChanged" KeyDown="CommandBox_KeyDown"
PlaceholderText="Type command..." BorderThickness="0"
Background="Transparent"
Foreground="#CDD6F4" Padding="12,10" FontSize="13"
FontFamily="Cascadia Code,Consolas,monospace"/>
</Border>
<Button Grid.Column="2" Click="RunCommand_Click" CornerRadius="10"
Height="40" Padding="14,0"
IsEnabled="{x:Bind CommandBox.Text.Length, Mode=OneWay,
Converter={StaticResource InverseBoolConverter}}"
BorderThickness="0">
<Button.Background>
<LinearGradientBrush StartPoint="0,0" EndPoint="1,0">
<GradientStop Color="#7C6AF7" Offset="0"/>
<GradientStop Color="#60A5FA" Offset="1"/>
</LinearGradientBrush>
</Button.Background>
<TextBlock Text="Run" Foreground="White" FontWeight="SemiBold"
FontSize="13"/>
</Button>
</Grid>
</Border>
</Grid>
</Page>
'@ -Encoding UTF8

```

```

Set-Content "$root\Views\TerminalPage.xaml.cs" @"
using Microsoft.Extensions.DependencyInjection;
using Microsoft.UI.Xaml;
using Microsoft.UI.Xaml.Controls;
using OllamaHub.ViewModels;
namespace OllamaHub.Views;
public sealed partial class TerminalPage : Page
{
public TerminalViewModel ViewModel { get; }

```


